
IRIDIS

ALPHA

THEORY

THE CODE OF A VIDEOGAME

IRIDIS ALPHA THEORY

For my beloved wife Edna.
Thank you for your love and understanding.

© Rob Hogan 2025, All Rights Reserved.
Edition Date: Saturday 1st November,
2025



This work is licensed under a Creative Commons
"Attribution-NonCommercial-ShareAlike 3.0 Unported"
license.



Contents

1	We Need to Talk About Binary	11
2	A Little Archaeology	18
2.1	The Madness Begins	20
2.2	After Our First Real Byte	23
2.3	A Loader for your Loader	27
2.4	Putting an End to the Madness	34
3	Some Disassembly Required	37
3.1	Important Concept Number One: High Bytes and Low Bytes	42
3.2	Important Concept Number Two: Interrupts	43
4	The First 16 Milliseconds	46
4.1	Sprites	48
4.2	Waiting for the Beam	50
4.2.1	Drawing the Stripes	53
4.2.2	Drawing the Text	56
4.3	Racing the Beam	60

4.3.1	A Complication	65
4.3.2	Back to the Beam	69
4.4	Enter The Gilbies	71
4.5	Title Text	75
5	Making Planets for Nigel	78
5.1	Step One: Creating the Sea	81
5.2	Step Two: Creating the Land	84
5.3	Step Three: Structures Structures Structures	88
5.4	Step Four: Add the warp gate	92
5.5	Inactive Lower Planet	93
5.6	Drawing the Lower Planet	94
5.6.1	Flipping the Byte	99
5.6.2	Flipping the Byte's Position	103
6	Blasting, Fast and Slow	107
6.1	Updating Enemy Sprite Positions	110
6.2	Scrolling the Planets	111
6.2.1	Pixel Movement	112
6.2.2	Character Movement	114
6.3	Jumping Up and Down	119
6.4	Sound Effects	124
6.4.1	The Sound Effect Data Structure	124
6.4.2	Multiplexing	129
6.4.3	Unused Sound Effects	134

7	Enemies and their Discontents	137
7.1	Something Simple - Byte 0: The Enemy's Color	139
7.2	Bytes 1-4: Sprite Animation	140
7.3	Bytes 18-21: Enemy Movement	144
7.3.1	What is going on with Byte 6?	146
7.4	Bytes 6-8: Alternate Enemy Waves	148
7.5	Bytes 22-23: The Bytes of Death	152
7.6	Byte 35: Energy Multiplier	154
7.7	Byte 34: Score for Hitting the Enemy	156
7.8	Bytes 9-11: A Clever Plan	157
7.9	Bytes 15-17: Sprite Switching	159
7.10	Bytes 28-29: Data to Load When Hit by a Bullet	161
7.11	Bytes 30-31: Data to Load After Colliding with the Player	163
7.12	Bytes 24-27 and 32-33: Enemy Phases	164
8	Congoatulations Hotshot	167
8.1	Entry Sequence	168
8.2	Generating Maps	172
8.2.1	Choosing a Map	180
8.3	Some Very Ugly Sprites	183
8.4	IBalls	183
9	A Hundred Thousand Billion Theme Tunes	185
9.1	Some Basics	188
9.1.1	Structure	190

9.1.2	Phrasing	195
9.1.3	Seeding the Random	200
10	Another 16⁴ Tunes	205
10.1	Taurus:Torus	207
10.2	A Graphical Interlude: Oscillator in 4 Parts	211
10.3	Taurus/Torus Two	219
11	Made in France	222
11.1	Main Loop	225
11.2	A Target Explodes	227
12	A Pause Mode for your Pause Mode	232
13	Iridis Oops!	241
13.1	The Byte that Broke	241
13.2	Reappearing Enemies	244
13.3	A Sort of Cheat	245
A	Jeff Minter's Development diary for Iridis Alpha	247
B	Sprite Atlas	266
C	Enemy Data	274
D	Planet Data	373
E	18/100,000,000,000,000 Theme Tunes	384

F Bonus Phase - Map Segments	405
G Bonus Phase - Map Rows	410
H Bonus Phase - Tilesheet	413
I Bumph	414
Index	419
Notes & References	430

About This Book

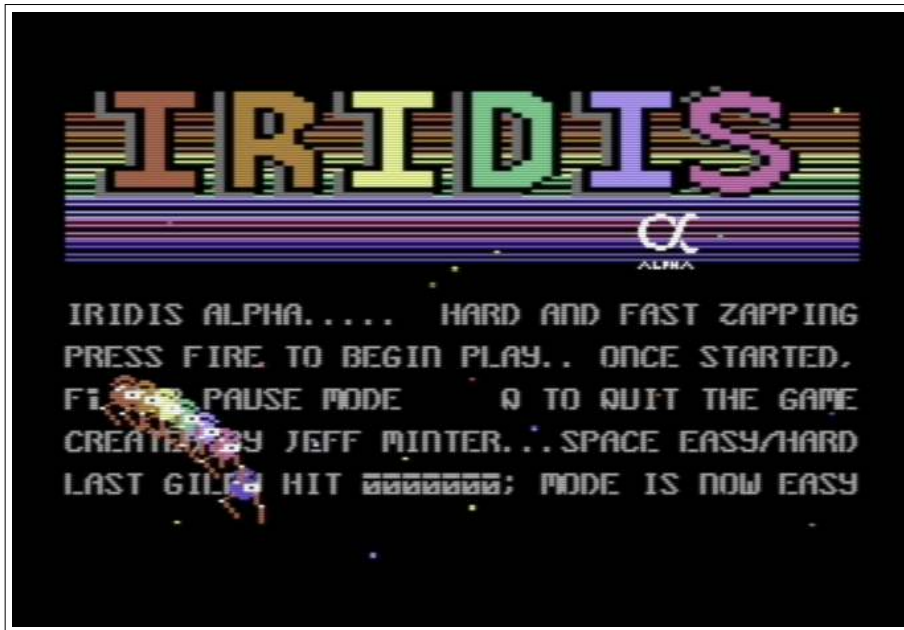
This book describes the inner workings of a relatively obscure video game created by an eccentric Englishman by the name of Jeff Minter in 1986 for the Commodore 64.

Iridis Alpha was not an enormous success on its release but it is widely regarded as one of the great achievements on Commodore 64 hardware. It embodies an 8-bit arcade aesthetic that was unique to its time, it was slightly mad in its concept and execution, and its emphasis on speed and unforgiving gameplay influenced generations of game developers in the years that followed.

But this book is not just the description of an old computer game. It is a tour through computer technologies that could justifiably be considered antique yet which are still relevant both to the creation of computer games and to the programming of computers in general.

Modern computer games can involve hundreds of people in their creation, but in 1986 the majority of games were conceived and executed by individuals working entirely on their own like Jeff Minter. The difference is not just one of scale but complexity. The detail that underlies a computer game from 1986 can fit between two covers with relative ease because the programmer was working directly with pixels on the screen (not to mention soundwaves from a speaker) and was constrained by physical limitations to achieve their goals using as few computer cycles and as little computer memory and storage as possible. Modern computer games on the other hand are a layer cake of infinitely dense detail we couldn't possibly hope to capture in their totality in a single book. With something as small as Iridis Alpha on the other hand, which is just 54,000 bytes, we can understand its operation at the lowest level and set out the challenges and solutions involved in making a very fast game on slow hardware. The techniques and tricks we discover are as important today as they were in 1986.

This book is also an introduction 'by example' to the lingua franca of all computers (even those we use today): assembly language. The flavour of assembly language we learn about here, 6502 Assembly, is no longer in contemporary use but it is one



The Iridis Alpha title screen.



Iridis Alpha: On the Brick Planet.

of the ancestors of the languages used in modern processors. Its relative simplicity and clarity makes it the ideal primer for learning the principles of programming in 'machine code' and understanding how computers work at a fundamental level. Strip away the abstraction and indirection provided by modern programming languages such as Python, Java, or even 'C', and you are left with enormous mounds of apparently indecipherable assembly language instructions. In this book I hope to show you that these instructions can be deciphered, and were once a viable way of programming computers.

Finally, this book cannot help being a very specific description of the way the Commodore 64 itself worked. The Commodore 64 was one of the most successful home microcomputers ever made. Along with the Apple II and the BBC Micro it was computing's first step into the home. Although they enabled new uses of computers, and marked the advent of video games in people's living rooms, these machines were still simple creatures. And because the Commodore 64 was such a basic computer in so many ways, what we end up learning about in this book is a recognisable implementation of a foundational concept in information theory: [the Turing Machine](#). 64 kilobytes of integrated circuit memory is almost all the C64 has to play with: there is no hard disk to speak of. So on our journey we find ourselves with a long Turing-style memory 'tape' on which our assembly language instructions operate, hopping back and forth along the 65,536 slots on the tape writing and reading values which the computer hardware translates into sound and vision. Understanding the code of a video game on an old computer, it turns out, is a great way to introduce ourselves to the practical details of computer science's first 'big idea'.

On the other hand, maybe you are just curious. Maybe you just want to learn the detailed mechanics of how to make a glorified digital breadboard such as the Commodore 64 produce something on a screen that flashes, bleeps, and fascinates the viewer. If so, then this book was written for you.

Overview of the Chapters

Since the source code of Iridis Alpha is no longer available, I have had to [unpack and reinterpret](#) it from the game binary Jeff Minter [released into the public domain in 2019](#). I describe this reverse-engineering process in the opening chapters because I think it is an interesting exercise in and of itself to go from a binary blob to a set of fully commented source code that provides insight to the inner workings of the game. [A Little Archaeology](#) describes how we extract the game binary from the cassette tape on which it was originally distributed. [Some Disassembly Required](#) shows you how to go from a very long list of bytes to a full source code listing. Hopefully you are here because you enjoy this kind of gory detail too.



Walking on the Forest Planet.



Near the stargates on the Brick Planet.

If you are just interested in learning about the mechanics of the game itself you can flick straight to [The First 16 Milliseconds](#), which begins by covering the loading the title screen and all that goes on in there. [Making Planets for Nigel](#), [Blasting Fast and Slow](#), and [Enemies and Their Discontents](#) cover the main gameplay and dive into how the game levels are created and how the speed of the game is achieved. These chapters go deep into the game's reassembled code and look closely at how Jeff Minter designed its core engine.

I dedicate a full chapter [A Hundred Thousand Billion Theme Tunes](#) to the ingenious procedural programming behind Iridis Alpha's theme tunes and trace its inspiration back to an article 'Musical Fractals' in a 1986 issue of Byte magazine. [Another 16⁴ Tunes](#) looks at the early experimentations with this fractal music in the demo program 'Torus' that Minter released while developing Iridis Alpha. It also delves into the animation experiment it contains, something that was used in the bonus screen in Iridis Alpha itself.

Iridis Alpha is full of little additional extras. [Congoatulations Hotshot!](#) unpacks the vertical scrolling mini-game Minter inserted as a bonus sequence. Even if the game itself is a little half-baked this bonus sequence is full of intrepid effects and some clever techniques for generating the game maps. [Made in France](#) and [A Pause Mode for Your Pause Mode](#) unpick the pause mode games bundled with the main game.

As a final coda, [Iridis Oops!](#) takes a look at some of the bugs that slipped through to the final release.

If you are reading the PDF version of this book the [Appendices](#) contain a data dump of sprite sheets, character sets, maps and tables that should provide hours of rewarding bedtime reading, or more likely, deep undisturbed sleep.

Note on the Text

If you find the writing hard going or the attempts to explain things difficult to follow, by all means [leave me a note](#) and I will gratefully accept your complaint. In the meantime, please skip over any blemishes to the next pretty picture or promising-looking block of text.

The full source code is available in [its own Github repository](#). You should find that it matches exactly the snippets of code provided in the book, though in some cases the extracts in the book have been edited and reformatted for brevity.

[Rob Hogan](#)

Dublin, 2023 - 2024

IRIDIS ALPHA THEORY

We Need to Talk About Binary

Here we are, ground zero in every book about assembly programming. Where the reader finds themselves in an awkward sit-down discussion about the B word: Binary. It's a necessary evil of course. Nothing will ever make sense without a rudimentary but slowly developing grasp of what bytes are and why they keep being referred to in a confusing mixture of letters and numbers such as \$D012 and \$F4.

So what am I supposed to do with you here? Succeed where every other author has usually failed? I assume that's the expectation. OK here goes.

You already know (OK *maybe* you already know) that computers, even those in the 1980s, understand everything in terms of 1s and 0s. All they know is a world in which something is on or off, high or low, present or absent. The reason for this is simply because yes/no and on/off are so fundamental to the reality of very small things. Once you break anything down far enough you are always left with a simple question of there being somewhere or nothing. The idea of a 1 or a 0 as the simplest possible building blocks for every single thing is not so outrageous. You put enough 1s and 0s together in a row you can build patterns that repeat and create larger patterns and suddenly you find yourself with a string of 1s and 0s that means something.

The idea of on/off is so simple even a computer can understand it. If you feed it a signal all it has to do is determine whether it is one or the other and store the result. Then read the next signal. And before long the computer has a sequence of 1s and 0s it might be able to do something with.

So when you are programming a computer or in our case trying to understand how it was programmed to give us Iridis Alpha you have to be able to visualize how the computer has stored all the ones and zeros it has been given and how it shuffles them around. An unruly mound of 1s and 0s is not much use when operating on them intensively. There has to be some benefit to splitting up the blob in some way that allows

some order to be imposed. Some way of segmenting a string of 1s and 0s that is both useful to us as the programmers and an efficient way of directing the computer to make effective use of the amorphous binary goo fed into it.

Trial and error has eventually arrived at an optimal arrangement, one which you have definitely heard of: this is the `byte`. The idea of the `byte` is simply to divide and conquer. We take any string of 1s and 0s and split them up into segments of eight. We now consider ourselves to have a string of bytes and we call each of the individual 1s and 0s `bits`.

Any scheme that reduces the number of items we have to deal with is a boon, but is there any rhyme or reason to choosing 8 as our magic number instead of say 7, or 12? Believe it or not, the number 8 was chosen (after much experimentation with others) almost solely because it proved the easiest and most convenient for people to deal with when understanding how they would make computers work.

The reason a byte of 8 bits is so convenient to deal with is because of the number of different values it can represent and the compact notation this particular number allows. Having a convenient notation system that allows humans to compose larger values from smaller values is surprisingly important when it comes down to it. It is humans doing the important part of the computing: making the big decisions, figuring out where things go and how they should work. The computer is just a glorified bit shuffler for which everything is on or off, it has no concept of a bigger picture. Humans need to be able to at least intuit some of this shuffling with a mental model of what happens when one set of ones and zeros is clashed with another. Using the magic number of 8 as the denominator for batches of bits enables this simple mental model.

The first great thing about 8 is that it can be divided into two. That is, into two segments of 4 bits. The great thing about a sequence of four 1s and 0s is that it has 16 possible permutations. That's to say it can be ordered in sixteen unique ways. Yet another way, is to say that a sequence of 4 bits can store up to sixteen different values.

Value	Bits	Value	Bits
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

The 16 different permutations of four 0s and 1s.

You have to agree this is marvellous stuff. But what is so convenient about a number between 0 and 15? If you're using decimal notation it's not convenient at all, in some cases you end up having to write two whole characters rather than one. We could avoid that if we invented a system that allowed a single character for each of the 16 values.

For example 0–9 for 0-9, and A–F for 10-15. Seems a bit clunky. But it works.

Value	Bits	Value	Bits
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

The 16 different permutations of four 0s and 1s but with letters too.

On the other hand, congratulations, we've just invented hexadecimal.

The genius of this system is that it allows us to write every possible sequence of 8 bits with just two characters.

Value	Bits	Value	Bits
00	00000000	80	10000000
01	00000001	81	10000001
02	00000010	82	10000010
...	...	...	...
7D	01111101	FD	11111101
7E	01111110	FE	11111110
7F	01111111	FF	11111111

The 256 different permutations of eight 0s and 1s, with a gap in the middle for reasons of space.

But that is not its only benefit. When we look at any specific number using this notation we can almost immediately tell how the bits in it are set, without needing to memorize all 256 possible permutations.

This is possible because we can start by looking at either character in the two-

character notation and quickly divine the 1s and 0s that make it up. We can do this because each character represents a value between 0 and 15 that is made up of some combination of 8, 4, 2, and 1 - each representing a bit from left to right.

8	4	2	1	Decimal	Hex
0	0	0	0	0	0
0	0	0	1	1	1
0	0	1	0	2	2
0	0	1	1	3	3
0	1	0	0	4	4
0	1	0	1	5	5
0	1	1	0	6	6
0	1	1	1	7	7
1	0	0	0	8	8
1	0	0	1	9	9
1	0	1	0	10	A
1	0	1	1	11	B
1	1	0	0	12	C
1	1	0	1	13	D
1	1	1	0	14	E
1	1	1	1	15	F

The 16 different permutations of four 0s and 1s but with letters too.

"All" we have to do when confronted with a single hexadecimal digit like 9 is figure out that its made up of 8 and 1 in order to know that the bits are set as follows:

8	4	2	1	Decimal	Hex
1	0	0	1	9	9

Likewise with the slightly more inscrutable D. Since F is 15, D must be 13. And 13 is made up of 8, 4, and 1:

8	4	2	1	Decimal	Hex
1	1	0	1	13	D

The ability to do this is especially important when doing the low level programming

that is typical of assembly. We'll see plenty of examples in future chapters but here is a taste of the kind of operation we ask a computer to perform by expressing our intentions using hexadecimal notation for binary.

Imagine we have a value from some random source stored in a byte. Imagine we happen to be in need of a random number, for example to select a random planet to use the next time we run attract mode in Iridis Alpha.

This random number we have stored in a byte sounds like it should be useful to us, but since its random it can be any value between 0 and 255 and since we only have four planets to choose from what we need is a number between 0 and 3.

A simple binary operation provides a way of solving this problem and our hexadecimal notation gives us a straightforward way of expressing it.

First the binary operation itself. Let's say our random value is a string of bits like 01011010 which is 5A in hexadecimal.

8	4	2	1	8	4	2	1	Decimal	Hex
0	1	0	1	1	0	1	0	90	5A

Since we only want a value between 0 and 3 we can extract out just the last two bits to give us that number. In other words, if we set all the bits to zero except the last two we would have a number between 0 and 3:

8	4	2	1	8	4	2	1	Decimal	Hex
0	0	0	0	0	0	1	0	2	02

We can ask the computer to do this for us by asking it to compare 01011010 with 00000011 and give us a result where all the bits in both are 1.

Byte	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
\$5A	0	1	0	1	1	0	1	0
\$03	0	0	0	0	0	0	1	1
Result: \$02	0	0	0	0	0	0	1	0

AND'ing \$5A and \$03 gives \$01 (1). For AND to give a 1 both bits must be 1.

Notice that in this new bit string we've set the bits that we're interested in to 1. This is called an AND operation. We have used 00000011 to mask out the bits we're not interested in and hence turn a random number between 0 and 255 into a random number between 0 and 3, in this case: 2.

Our choice of 8 as a magic number becomes slightly less magical when we realize that we never had all possible numbers to choose from in the first place. Sure we could have chosen 7 or 9 or 13 but how practical would this really be when the actual value assigned to any individual bit is always going to be a power of 2. Even if we could never precisely intuit the reasons as to why, a magic number that is also the power of 2 (e.g. 8, 4, 16, 64) is always going to make things easier over the long haul even if we can't articulate them.

So now some jargon we might have heard in the past is starting to make sense. The Commodore 64 was an '8-bit computer'. What this means is that its basic unit of currency is the 8-bit byte we've invented here. Every time it performs an operation it does so using a single byte as its fundamental building block. This is true whether we're talking about the AND operation or storing a value for use later on. When we give the C64 a value it's always a single byte. The subsequent era of 16-bit computing (e.g. the Amiga) took this a step further by introducing a two-byte block as its basic unit of currency. 32-bit and 64-bit computing are the *lingua franca* of modern processors - no matter how small the value we're using it will be managed as 4-byte or 8-byte value respectively by 32-bit and 64-bit computer architectures.

But even the C64 had use for values larger than 255. In fact it could handle staggering numbers as large as 65,535 using a little sleight of hand to put two bytes side by side and treat them as a single number.

This was as far as its ken could reach however. And since it could look no further than 65,535 that imposed an upper bound on the number of values it could reach for at any given time. This finite horizon is known as its address space. When we want to store a value in the C64's memory we only have 65,536 slots available to specify or select from. The address we choose must be between \$0000 (0) and \$FFFF (65,535).

43	36	34	00	40	..	41	50	45	DE	A2
\$0000	\$0001	\$0002	\$0003	\$0004	..	\$FFFC	\$FFFC	\$FFFD	\$FFFE	\$FFFF

Figure 1.1: The C64's memory visualized as a long strip of 65,536 slots, each holding a byte represented in hexadecimal.

This upper limit is what gives the Commodore 64 its name. It has 64 kilobytes of memory.

Hex	Decimal	Meaning
\$0400	1024	1 Kilobyte
\$FFFF	65,535	64 Kilobytes

Some magic numbers when dealing with bytes.

This is all we need to get started. The rest we can pick up along the way. So let's start.

A Little Archaeology

Iridis Alpha was distributed on cassette tape by the publisher Hewson Consultants. Normally used to play audio, cassette tapes were the cheap and ubiquitous medium *du jour* of the 1980s and a natural choice for the nascent 8-bit game industry to distribute its wares.

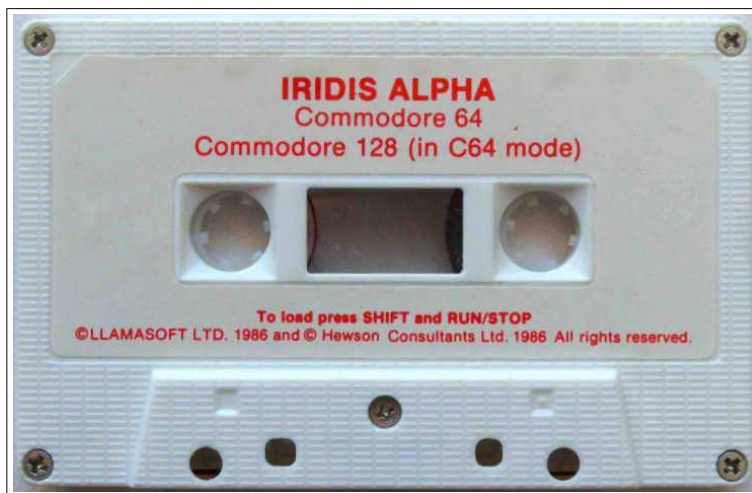


Figure 2.1: It should be simple getting bytes off this right?

You don't have to look at the picture of Iridis Alpha's cassette tape above to notice that it doesn't contain an awful lot of tape. This gives you an idea of how little data we are actually dealing with when it comes to the size of program that the C64 could cope with. The spool contains a 'mere' 54 kilobytes of target data. Even when encoded in the elaborate way we are about to describe this still does not result in a large physical

medium. As you may already appreciate, 54 kilobytes is a fraction of the average file size we are used to dealing with today, where even a single image on a web page will run to hundreds of kilobytes.

Playing a cassette tape for a C64 game such as Iridis Alpha on a normal cassette tape player would be an ear-splitting mistake. Instead of music you would be subjected to a cacophony of mechanical chirruping. This is the tape attempting to convey to you its long stream of binary data in the only language available to it: lots of sound waves of varying length. The Commodore 64 came equipped with a specially-adapted cassette player of its own to cope with this racket. This player could read the sounds encoded on a cassette tape and transmit them as a series of pulses into the C64's main CPU board, so that they could be used for their intended purpose: interpretation as bytes of data.

Without knowing how it's actually done, it's tempting to imagine a variety of possible schemes that might have been used to store data on a cassette. For example, one sound wave denoting a '0' and another one denoting a '1'. The method that is actually used isn't very far away from such a thing but there is plenty of intricacy layered on top, particularly in a bid to spend as little time as possible loading data from the tape.

In order to get something to work with on our modern computers, we must first somehow convert the contents of the cassette tape into a binary file on our computer's hard disk and then emulate the steps the C64's tape player performed to read the sounds from the tape. Only then can we load them into memory as something that could be run as a computer program.

The earliest and most durable attempt at standardizing this procedure was from Per Hakan Sundell in 1997. He invented what is now known as the 'tap' format for representing the beeps and bleeps encoded on the tape as a file of bits and bytes. The idea is that each byte in the tap file represents the length of a pulse encoded on the tape. It is the length of these pulses that will ultimately tell us whether we should interpret a value of 1 or 0. When we get eight 1s and 0s we have a byte. We get enough bytes, we have a program we can run!

Someone, somewhere has kindly decoded the contents of the Iridis Alpha cassette tape distribution to a tap file for us. So we have something to dig into. This is going to be a slightly bonkers journey into the bowels of decoding a 54KB game file from over 500KB of raw data. Every time you think you are nearly done there will be yet another convolution to wrap your head around. But at the end of it we will finally have our binary game file and will be ready to figure out how to decipher it into something approximating the original 6502 assembly language Jeff Minter used to write Iridis Alpha in 1986.

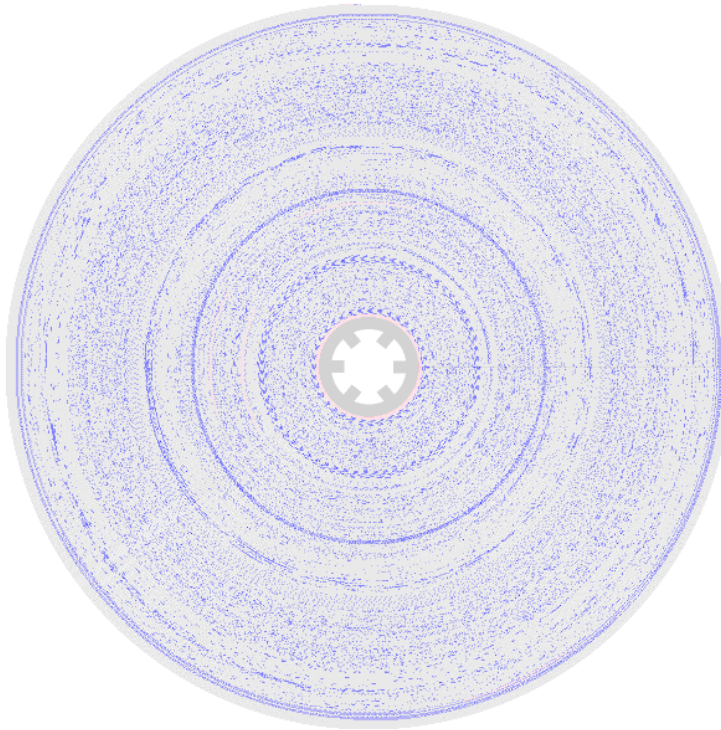


Figure 2.2: Simple, right? In this visualisation of a fully wound cassette spool containing Iridis Alpha each short-pulse sound is represented by a light grey pixel, each medium-duration pulse by a blue pixel, and each long pulse by a pink pixel. Roughly speaking, grey is a shorthand for 0 bits and dark pixels for 1 bits.

2.1 The Madness Begins

In the table below we show what the start of the our `iridis-alpha.tap` file looks like. Note that this table presents the the contents of the start the of `tap` file rather than the data on the tape itself. The data on the tape is represented by the white cells in our diagram. The coloured cells are header information specific to the `tap` file format and do not represent data on the tape. The purpose of this header info is make it obvious to anyone inspecting `iridis-alpha.tap` that it contains raw sound information captured from a cassette (C64-TAP-RAW), as well as providing the version of the `tap` specification used when creating the file and the total length of the data in the file:

43	36	34	2D	54	41	50	45	2D	52	41	57	00	00	00	00
5A	0A	08	00	00	5D	32	2F	30	2F	2F	30	30	31	30	31
30	31	31	2F	31	31	30	31	30	30	31	30	30	30	31	30
31	31	30	31	31	30	31	30	30	31	30	31	31	30	30	30
31	31	31	30	30	31	30	31	31	31	30	30	30	31	31	30
30	30	31	30	31	30	30	30	31	30	31	31	30	30	30	31
31	31	30	30	31	30	32	31	30	31	30	30	32	31	30	30

Figure 2.3: The first batch of data we read in from the 'tap' file. The 'Header Description' is in grey, the Version Number in pink, Reserved bytes in blue, and the File Size in red.

Field Description	Field Value	Note
TAP Format Header Description	43 36 34 2D 54 41 50 45 2D 52 41 57	'C64-TAPE-RAW' in ASCII
Version	00	Version Number 0
Reserved	00 00 00	Used by format versions 0
File Size	5A 0A 08	\$080A05, in decimal: 526,853 bytes long.
Start of Data	00 00 5D 32 2F 30 2F 30 30 31 30 31	Bytes representing the individual pulses/sounds on the tape.
	30 31 31 2F 31 31 30 31 30 30 31 30 30 30 31 30	
	31 31 30 31 31 30 31 30 30 31 30 31 31 30 30 30	
	31 31 31 30 30 31 30 31 31 31 30 30 30 31 31 30	
	30 30 31 30 31 30 30 31 30 31 31 30 30 30 31	
	31 31 30 30 31 30 32 31 30 31 30 30 32 31 30 30	

Figure 2.4: The meaning of the first batch of data we've read in.

After the header information described above, each byte in the `tap` file represents the pulse length or duration of a single sound emitted by the tape. A pair of sounds taken together represent a single bit, i.e. a 0 or a 1. A medium length sound followed by a short length represents a 1, a short length followed by a medium length represents a 0.

The following table shows us whether we should consider a byte on the tape to represent a short, medium, or long duration sound:

Sound Length	Lower Range	Upper Range
Short	\$24	\$36
Medium	\$37	\$49
Long	\$4A	\$64

Figure 2.5: Values for short, medium, and long pulses. For example, any byte value on the `tap` file between \$24 and \$36 would be considered a 'Short' pulse.

Remarkably the first 27,000 or so pulses on the Iridis Alpha tape are nothing but short sounds (values between \$2F and \$31) so cannot be interpreted as anything. These would have been ignored by the C64 when reading the tape and so can be ignored by us. It's not until the 27,157th byte in the tape that we start to encounter real data:

```
00006a10: 3031 3031 3156 4144 3130 4231 4243 3130 01011VAD10B1BC10
00006a20: 4231 4131 4244 312f 4057 4032 4231 4231 B1A1BD1/@W02B1B1
00006a30: 4244 312f 4231 4231 4243 3142 3055 4144 BD1/B1B1BC1B0UAD
00006a40: 3142 3143 3130 4231 4231 4232 4244 3142 1B1C10B1B1B2BD1B
00006a50: 3055 4131 4243 3142 3130 4231 4231 4231 0UA1BC1B10B1B1B1
00006a60: 4244 3130 4056 4143 3230 4143 3130 4231 BD10@VAC20AC10B1
00006a70: 4231 4232 4244 312f 4056 4232 4231 4243 B1B2BD1/@VB2B1BC
00006a80: 3230 4231 4231 4231 4243 3142 2f54 4245 20B1B1B1BC1B/TBE
00006a90: 3141 3130 4231 4231 4230 4243 3030 1A10B1B1B0B0BC00
```

Listing 2.1: Data finally gets started at '56 41' in the first line above.

The first meaningful twenty bytes therefore are:

```
56 41 44 31 30 42 31 42 43 31 30 42 31 41 31 42 44 31 2F 40
```

You get a sense of how wasteful, or ahem redundant, this encoding scheme is when you learn that these twenty pulses are required to give us a single byte. The table below shows how we interpret them to construct a series of 1s and 0s.

First Byte	Second Byte	First Byte Pulse Length	Second Byte Pulse Length	Meaning
\$56	\$41	Long	Medium	Start of Byte Indicator
\$44	\$31	Medium	Short	\$01
\$30	\$42	Short	Medium	\$00
\$31	\$42	Short	Medium	\$00
\$43	\$31	Medium	Short	\$01
\$30	\$42	Short	Medium	\$00
\$31	\$41	Short	Medium	\$00
\$31	\$42	Short	Medium	\$00
\$44	\$31	Medium	Short	\$01
\$2F	\$40	Short	Medium	Parity Bit of \$00

Figure 2.6: Interpretation of the first 20 meaningful bytes creating a byte. The parity bit at the end is a \$00 if there are an odd number of 1s, and \$01 if there are an even number of 1s. 10010001 has an odd number of 1s.

We can visualize the twenty bytes as a square sound wave. When reading the tape the C64 would interpret these sound pulses as long, short, or medium to construct a meaning for the entire sequence.

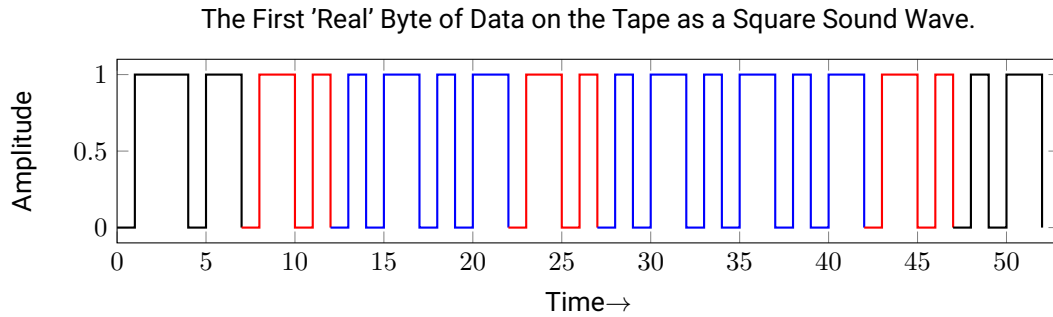


Figure 2.7: Medium-Short pairs in red represent a '1' bit, Short-Medium pairs in blue represent a '0' bit. So this gives us '10010001'. The black wave form at the beginning is the 'Start of Byte' indicator and the one at the end is a parity bit.

Take your time to compare the wave diagram above with the table on the opposite page. We're representing the same data in two different ways here, with the wave pattern illustrating the sound form that the cassette reader would have received and translated into the zeros and ones that will make up our first byte.

Once we've extracted our result of 10010001 from these twenty bytes we now must reverse it. We must do this because the bits are encoded on the tape with the 'most significant bit' first and we are used to reading binary with the 'least significant bit first'. In hexadecimal the reversed bit-string of 10001001 is \$89.

We have our first byte! It's \$89!

2.2 After Our First Real Byte

With this precious commodity in hand we now continue reading off bytes in the same manner from the tape. Eventually we encounter a signal that tells us we've reached the end of the data block, a 'Long-Short' sequence. When this happens we find we've read 202 bytes in total:

89	88	87	86	85	84	83	82	81	03	A7	02	04	03	49	52
49	44	49	53	00	00	00	00	00	00	00	00	00	00	78	A9
6E	8D	06	DD	A2	01	20	D4	02	26	F7	A5	F7	C9	63	D0
F5	A0	64	20	E7	03	C9	63	F0	F9	C4	F7	D0	E8	20	E7
03	C8	D0	F6	C9	00	F0	D6	20	E7	03	99	2B	00	99	F9
00	C8	C0	0A	D0	F2	A0	00	84	90	84	02	20	E7	03	91
F9	45	02	85	02	E6	F9	D0	02	E6	FA	A5	F9	C5	2D	A5
FA	E5	2E	90	E7	20	E7	03	C8	84	C0	58	18	A9	00	8D
A0	02	20	93	FC	20	53	E4	A5	F7	45	02	05	90	F0	03
4C	E2	FC	A5	31	F0	03	4C	B9	02	A5	32	F0	03	6C	2F
00	20	33	A5	A2	03	86	C6	BD	F3	02	9D	76	02	CA	D0
F7	4C	E9	02	A9	07	85	F8	20	D4	02	26	F7	EE	20	D0
C6	F8	10	F4	A5	F7	60	00	00	E4						

Figure 2.8: The data we’ve read in so far. The unshaded section is machine code.

Fortunately this data has a meaning. It contains the first part of a machine code program the C64 can execute:

Field Description	Field Value	Note
Countdown	89 88 87 86 85 84 83 82 81	Data Block Header
File Type	03	03=PRG, i.e. executable machine code.
Load Address	A7 02	Address to load to: \$02A7
End Address	04 03	End Address to load to: \$0304
Filename	49 52 49 44 49 53 00 00 00 00 00 00 00 00 00 00	Filename: "IRIDIS"
Machine Code	78 A9 6E 8D 06 DD A2 01 20 D4 02 26 F7 A5 F7 C9 63 D0 F5 A0 64 20 E7 03 C9 63 F0 F9 C4 F7 D0 E8 20 E7 03 C8 D0 F6 C9 00 F0 D6 20 E7 03 99 2B 00 99 F9 00 C8 C0 0A D0 F2 A0 00 84 90 84 02 20 E7 03 91 F9 45 02 85 02 E6 F9 D0 02 E6 FA A5 F9 C5 2D A5 FA E5 2E 90 E7 20 E7 03 C8 84 C0 58 18 A9 00 8D A0 02 20 93 FC 20 53 E4 A5 F7 45 02 05 90 F0 03 4C E2 FC A5 31 F0 03 4C B9 02 A5 32 F0 03 6C 2F 00 20 33 A5 A2 03 86 C6 BD F3 02 9D 76 02 CA D0 F7 4C E9 02 A9 07 85 F8 20 D4 02 26 F7 EE 20 D0 C6 F8 10 F4 A5 F7 60 00 00	This is the machine code of the program to execute. We will translate this back to assembly so we can understand what it does later.
Checksum	E4	I don't know how is this calculated!?

Figure 2.9: The meaning of the data we’ve read in so far.

This small program, once we have loaded the rest of it, is where the fun starts. But before we do that we have lots more busywork to do. How about reading in all of the

above data again from the tape? Yup, that's correct. As we keep reading the tape we will find that it contains all of the above data a second time, with the slight difference that the 'Data Block Header' will be 09 08 07 06 05 04 03 02 01 instead of 89 88 87 86 85 84 83 82 81.

We have to read another 23,000 or so more pulses before we get to something new that we're interested in, the second and final part of the program that we can execute.

When it arrives it looks like this:

89	88	87	86	85	84	83	82	81	A9	80	05	91	4C	EF	F6
A9	A7	78	8D	28	03	A9	02	8D	29	03	58	A0	00	84	C6
84	C0	84	02	AD	11	D0	29	EF	8D	11	D0	CA	D0	FD	88
D0	FA	78	4C	51	03	AD	0D	DC	29	10	F0	F9	AD	0D	DD
8E	07	DD	4A	4A	A9	19	8D	0F	DD	60	20	8E	A6	A9	00
A8	91	7A	4C	74	A4	52	D5	0D	00	00	00	00	00	00	00
00	00	8B	E3	AE	02	53									

Figure 2.10: The second part of our program in machine code. The unshaded section is the machine code.

Field Description	Field Value	Note
Countdown	89 88 87 86 85 84 83 82 81	Data Block Header
Machine Code	A9 80 05 91 4C EF F6 A9 A7 78 8D 28 03 A9 02 8D 29 03 58 A0 00 84 C6 84 C0 84 02 AD 11 D0 29 EF 8D 11 D0 CA D0 FD 88 D0 FA 78 4C 51 03 AD 0D DC 29 10 F0 F9 AD 0D DD 8E 07 DD 4A 4A A9 19 8D 0F DD 60 20 8E A6 A9 00 A8 91 7A 4C 74 A4 52 D5 0D 00 00 00 00 00 00 00 00 8B E3 AE 02	This is the rest of machine code of the program to execute.
Checksum	53	I stil don't know how is this calculated!

Figure 2.11: The meaning of the second batch of data we've read in.

Would you be surprised to learn that we have to read in this payload a second time from the tape before we're done? Let's just assume that you're not and let's move swiftly on to wondering what we're supposed to do with 100 or so bytes of data we've finally managed to read after listening to some 50,000 chirrups and clicks from cassette tape.

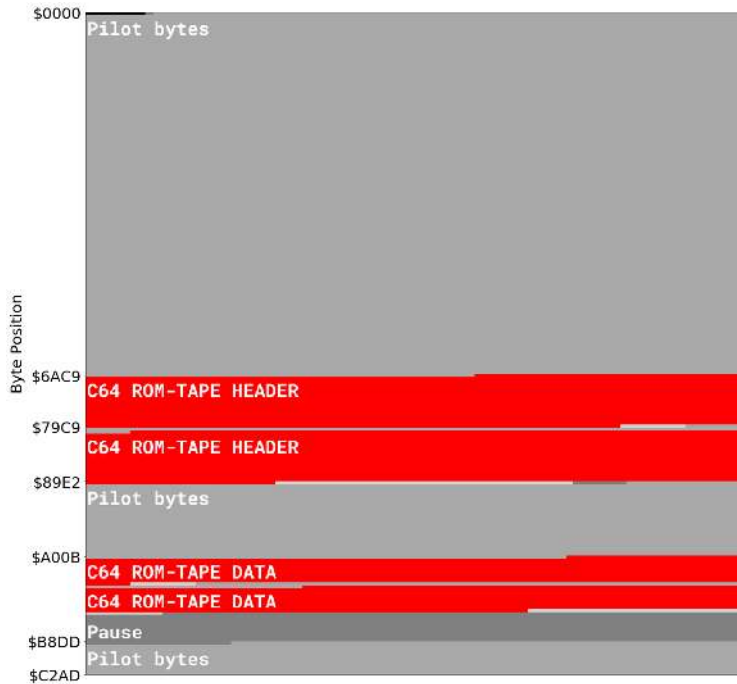


Figure 2.12: The bytes we've read so far from the tape.

The answer is simple. We load the data we've received into RAM and execute it. We load the second chunk of data we received at address \$02A7 (this was given in the 'Load Address' field) and the first chunk of data we received directly after it.

What is this program? It seems a bit short to be Iridis Alpha right? What it is a whole new program for reading the rest of the data from the tape. That's right: we've read all this data from the tape to get a program for reading data from the tape. This type of program is called a 'loader', or perhaps in an effort to justify its existence, a 'turbo loader'.

2.3 A Loader for your Loader

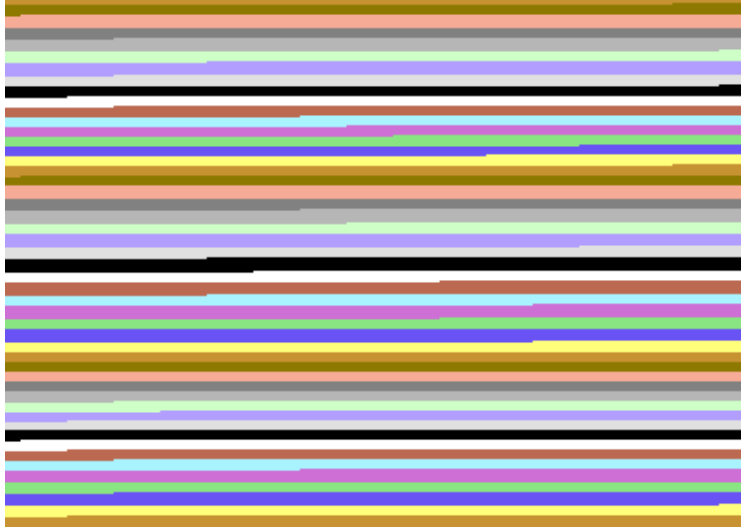


Figure 2.13: MegaSave loader pictured.

The idea is that this little program will do a better job of reading data from the tape and do it more quickly than the C64 can manage by itself. There is a whole menagerie of these programs that proliferated in the 1980s with exotic names such as Jetload, Easytape, Audiogenic and so on. Luigi di Fraia maintains a utility called `tapclean` that does a great job of identifying and emulating these loaders and thanks to him we have a disassembled version of the loader we've just found on the Iridis Alpha tape. It has the quintessentially 1980s moniker 'MegaSave' and as you can see in the listing reproduced on the next page it is relatively compact.

Now before you turn the page, bear in mind that you are not expected to read or understand this listing yet. I've reproduced it here so you can get a sense of what a small assembly program looks like. If you're going to read your first serious piece of assembly this is not where you should start. If anything else, the font is way too small!

```

; This disassembly has been adapted from
; https://www.luigidifraia.com/doku/doku.php?id=
;   commodore:tapes:loaders:mega-save
;
; From: Cauldron
; Note: Assemble with 64tass
;
; *****
; * CBM Header *
; *****

**=$033C

; Cassette I/O Buffer - Header
T033C
.byte $03,$A7,$02,$04,$03,$49,$52,$49,$44,$49,$53,$00,
    $00,$00,$00,$00
T034C
.byte $00,$00,$00,$00,$00,$00

AlignAndSynchronizeLoop
    SEI

    LDA #$6E      ; Set the read bit timer/counter
                  ; threshold to 0x0107
    STA $DD06
    LDX #$01

; Byte-align with the incoming stream by shifting bits
; in until
; the first pilot byte is read

_align
    JSR rd_bit    ; Read a bit
    ROL $F7       ; Shift each of them into the byte
                  ; receive register
    LDA $F7
    CMP #$63      ; Until the first occurrence of 0x63 (
                  ; pilot byte)
    BNE _align

    LDY #$64      ; Pre-load the first expected sync byte
                  ; in Y

; Read the whole pilot sequence

_pilot
    JSR rd_byte    ; Keep reading bytes until the pilot
                  ; train is over
    CMP #$63
    BEQ _pilot

; Check that the sync sequence is as expected (Note: $F7
; = byte read by
; means of a call to rd_byte)

_sync
    CPY $F7       ; Is the currently expected sync byte
                  ; following?
    BNE _align    ; Start over if not

    JSR rd_byte    ; Read byte

    INY           ; Bump the expected sync byte value
    BNE _sync     ; Read the whole sync sequence (0x64-0
                  ; xff inclusive)

    CMP #$00      ; In absence of issues A is set to 0x01
                  ; here
    BEQ AlignAndSynchronizeLoop

; Read and store file header
_header
    JSR rd_byte    ; Read 10 header bytes
    STA $002B,Y    ; Overwrite BASIC program pointers
    STA $00F9,Y    ; Also store them in RAM where they will
                  ; be changed
    INY
    CPY #$0A
    BNE _header

    LDY #$00
    STY $90       ; Zero the status flags (not set by this
                  ; code)
    STY $02       ; Zero the checksum register

```

```

; Read and store data from tape into RAM

_data
    JSR rd_byte    ; Read a byte
    STA ($F9),Y    ; Store in RAM

    EOR $02       ; Update the checksum value
    STA $02

    INC $F9       ; Bump the destination pointer
    BNE _check_complete
    INC $FA

_check_complete
    LDA $F9       ; And check if the file is complete
    CMP $2D       ; by comparing the destination pointer
    LDA $FA       ; to its one-past-end value
    SBC $2E
    BCC _data

    JSR rd_byte    ; Read a byte

    INY
    STY $C0       ; Control motor via software

    CLI

    CLC
    ; Make sure the call to $FC93 does not restore the
    ; standard IRQ
    LDA #$00
    STA $02A0
    ; Disable interrupts, un-blank the screen, and stop
    ; cassette motor
    JSR $FC93

    JSR $E453      ; Copy BASIC vectors to RAM

    LDA $F7       ; Compare saved and computed checksums
    EOR $02
    ; And check that the status flags do not indicate an
    ; error
    ORA $90
    BEQ **+5

    JMP $FCE2      ; Soft-reset if any of the above checks
                  ; fails

; Code execution after a file is completely loaded in
    LDA $31       ; Check flag #1
    BEQ **+5      ; If not set move on
    JMP J02B9      ; Otherwise re-execute the loader

    LDA $32       ; Check flag #2
    BEQ **+5      ; If not set move on to issue a BASIC
                  ; RUN command
    JMP ($002F)    ; Otherwise execute routine pointed by
                  ; the vector $2f/$30

    JSR $A533      ; Relink lines of tokenized program text

    LDX #$03      ; Set the number of characters in
                  ; keyboard buffer to 3
    STX $C6

    LDA T02F4-1,X ; And copy R, <shift> + U, <return> into
                  ; the buffer
    STA $0276,X
    DEX
    BNE **+7

    JMP J02E9

; -----
; Read byte: read 8 bits from tape, grouping them MSbF
; Returns: the read byte in A

rd_byte
    LDA #$07      ; Prepare for 8 bits
    STA $F8       ; Using $f8 as a counter

B03EB
    JSR rd_bit    ; Read a bit

```



```

; Shift each of them into the byte receive register,
MSBF
ROL $F7
INC $D020
DEC $F8          ; And loop until 8 bits are received
BPL B03EB
LDA $F7          ; Return read byte in A
RTS

.cerror * > $03FC, "The CBM Header code is too long to
               fit in the tape buffer!"
.align $03FC, $00 ; Padding

; *****
; * CBM Data *
; *****

**=$02A7

NMIH
LDA #$80
ORA $91
JMP $F6EF

J02AE
LDA #<NMIH

SEI

STA $0328        ; Disable <Run Stop> + <Restore>
LDA #>NMIH
STA $0329

J02B9
CLI

LDY #$00
STY $C6          ; No char in keyboard buffer
STY $C0          ; Enable tape motor
STY $02          ; Zero the checkbyte register (also done
                 later on)

LDA $D011        ; Blank the screen
AND #$EF
STA $D011

DEX              ; Wait a bit
BNE --1
DEY
BNE --4

SEI

JMP AlignAndSynchronizeLoop ; Execute the main
                           loader code

```

```

; -----
; Read bit: loops until a falling edge is received on
; the read line and uses the read bit timer/counter
; to discriminate the current bit value
;
; Returns: the read bit in the Carry flag

rd_bit
LDA $DC0D        ; Loop until a falling edge is detected
AND #$10         ; on the read line of the tape port
BEQ rd_bit

LDA $DD0D
STX $DD07
LSR
LSR              ; Move read bit into the Carry flag

LDA #$19         ; Restart the bit read threshold timer/
counter
STA $DD0F

RTS

J02E9
; Reset pointer to current text character to the
; beginning of program text
JSR $A68E

LDA #$00
TAY
STA ($7A),Y

JMP $A474        ; Print READY

; -----
; Characters to be injected in the keyboard buffer, if
; required

T02F4
.byte $52,$D5,$0D ; R, <shift> + U, <return>

.cerror * > $0300,
"The CBM Data code is too long to fit in front of the
vector table!"

; -----
; Overwrite BASIC vectors in RAM

.align $0300, $00 ; Padding

T0300
.word $E38B ; Leave IERROR unchanged
; Autostart the turbo loader, once loaded, by
overwriting IMAIN
.word J02AE

```

What does MegaSave do that makes it so much faster than the default C64 tape loader? The simple answer is that it cuts corners and strips away all of the cautious redundancy we observed when loading the loader itself. Instead of reading 20 bytes (or pulses) from the tape in order to construct a single byte it just needs 8. It does what we naively thought at the beginning might be the way to read data from the tape: a long pulse is a 0, a short pulse is a 1, you read 8 of them you have the 8 bits for your byte.

This simplicity is risky. With no repetition of data blocks, no parity check on each individual byte, and no delimiters between bytes, the loader is vulnerable to corruption of the tape itself or any hardware flakiness in the cassette reader. The fact that it generally works is simply due to a lack of conservatism paying off in practice. In addition, the MegaSave loader isn't totally bereft of precautions. There is a slightly elaborate

dance it goes through to gain some assurance that the tape medium is going to yield a reliable string of bytes.

The first thing it does is look for a sentinel value of \$63.

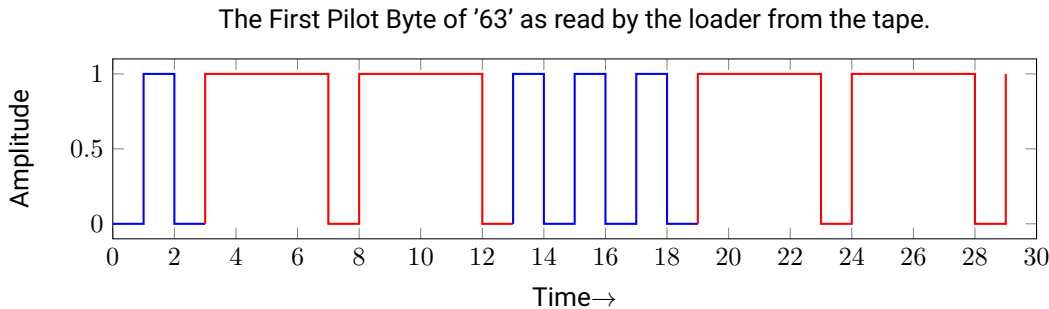


Figure 2.14: Long square waves in red represent pulses giving a '1' bit, Short square waves in blue represent a '0' bit. So this gives us '01100011' i.e. \$63. Unlike the default tape loader, MegaSave expects the 'most significant bit first' - which is the natural way of representing bits on paper so we don't need to reverse the bits to 'make sense' of them.

There is an inordinately long string of these, followed by an ascending sequence of byte values from \$63 to \$FF. The long series of \$63s are known as 'Pilot Bytes'.

63	63	63	63	63	63	63	63	63	63	63	63	63	63	63	63
63	63	63	63	63	63	63	63	63	63	63	63	63	63	63	63
63	63	63	63	63	63	63	63	63	63	63	63	63	63	63	63
63	63	63	63	63	63	63	63	63	63	63	63	63	63	63	63
63	63	63	63	63	63	63	63	63	63	63	63	63	63	63	63
63	63	63	63	63	63	63	63	63	63	63	63	63	63	63	63
63	63	63	63	63	63	63	63	63	63	63	63	63	63	63	63
63	63	63	63	63	63	63	63	63	63	63	63	63	63	63	63
63	63	63	63	63	63	63	63	63	63	63	63	63	63	63	63
63	63	63	63	63	64	64	65	66	67	68	69	6A	6B	6C	6D

Figure 2.15: The leading material read by the Megasave loader in the run up to retrieving game data. The Pilot Bytes are in red, the beginning of the 'Sync Train' in blue.

Following the pilot bytes we get an ascending series from \$63 to \$FF. This sequence

has the catchy name of a 'Sync Train'. Once the Sync Train has reached \$FF the decoder knows that its time to start taking some of this data seriously:

63	63	63	63	63	64	64	65	66	67	68	69	6A	6B	6C	6D	6E
6F	70	71	72	73	74	75	76	77	78	79	7A	7B	7C	7D	7E	7F
80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F	90
91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F	A0	A1
A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	B0	B1	B2
B3	B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	C0	C1	C2	C3
C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF	D0	D1	D2	D3	D4
D5	D6	D7	D8	D9	DA	DB	DC	DD	DE	DF	E0	E1	E2	E3	E4	E5
E6	E7	E8	E9	EA	EB	EC	ED	EE	EF	F0	F1	F2	F3	F4	F5	F6
F7	F8	F9	FA	FB	FC	FD	FE	FF	01	00	08	FF	BF	00	00	01
02	CA	00														

Figure 2.16: The rest of the leading material read by the Megasave loader in the run up to retrieving game data. The last few Pilot Bytes are in red, the 'Sync Train' in blue, and the Data Header fields from the grey cell onwards.

The bytes after the blue cells above are our first bit of raw meat in a while. Here's what they mean:

Field Description	Field Value	Note
Header Sentinel	01	Indicates the start of the header, expected to be non zero.
Load Address	00 08	Address to load to: \$0800
End Address	FF BF	End Address to load to: \$BFFF
Execution Address	00 00	Filename: "IRIDIS"
Next Action Indicator	01	Resume loading data when done or execute the loaded data.
Execution Type	02	How to execute the code.

Figure 2.17: The meaning of the Data Header values read in by MegaSave.

What the loader can garner from this is that the data that follows should be read in and stored at \$0800 and that rather than execute it straight away it should then resume loading more data from the tape.

The entire game is stored across four separate chunks on the tape. Once it has loaded this first one, the loader reads in the next three chunks.

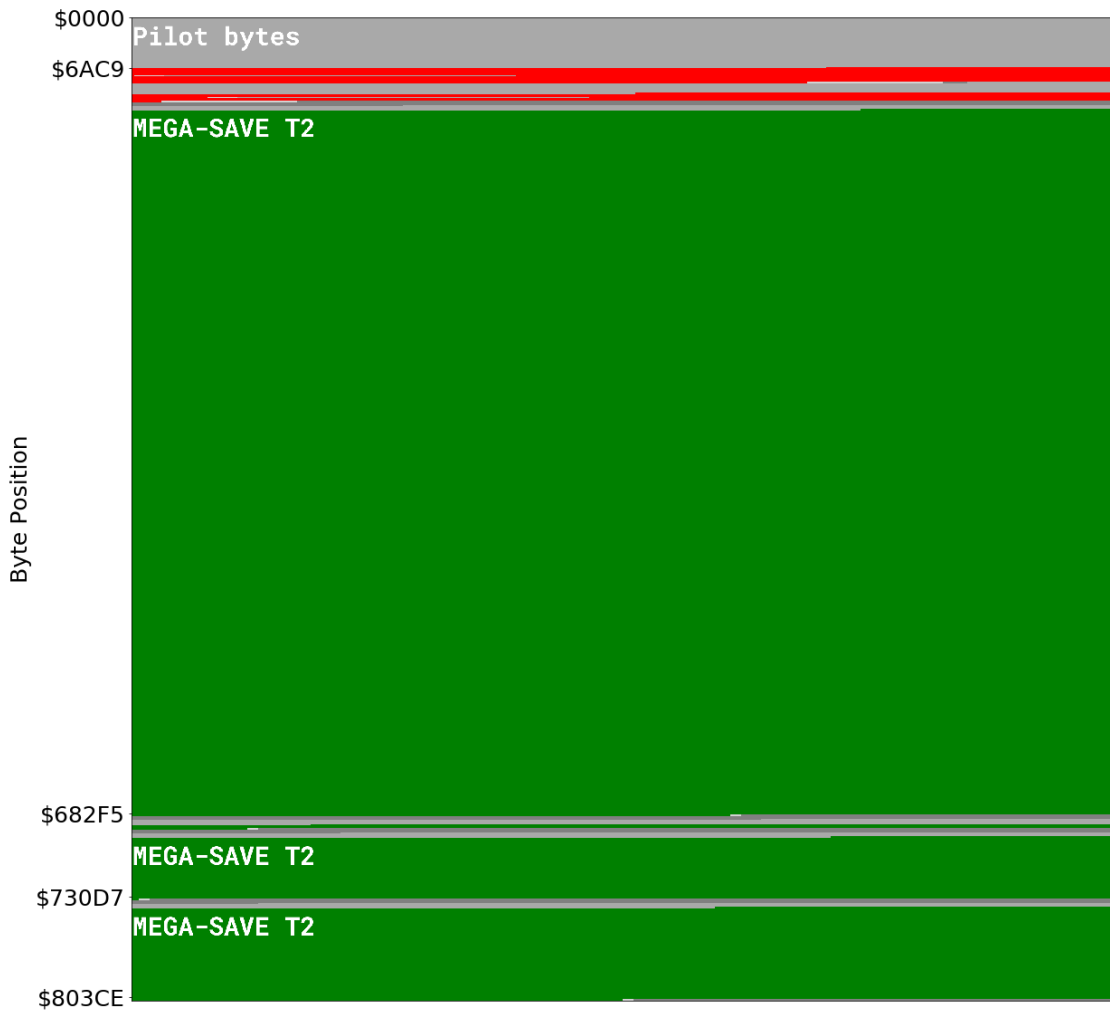


Figure 2.18: All the data that has been read from the tape. The four chunks of game data are in green the second is only a sliver visible below the label \$682F5. The relative sizes of the red data (the MegaSave loader which is only actually 200 or so bytes long) and the green data (representing over 50,000 bytes of game data) illustrate how efficient the MegaSave loader's storage is by comparison with the default.

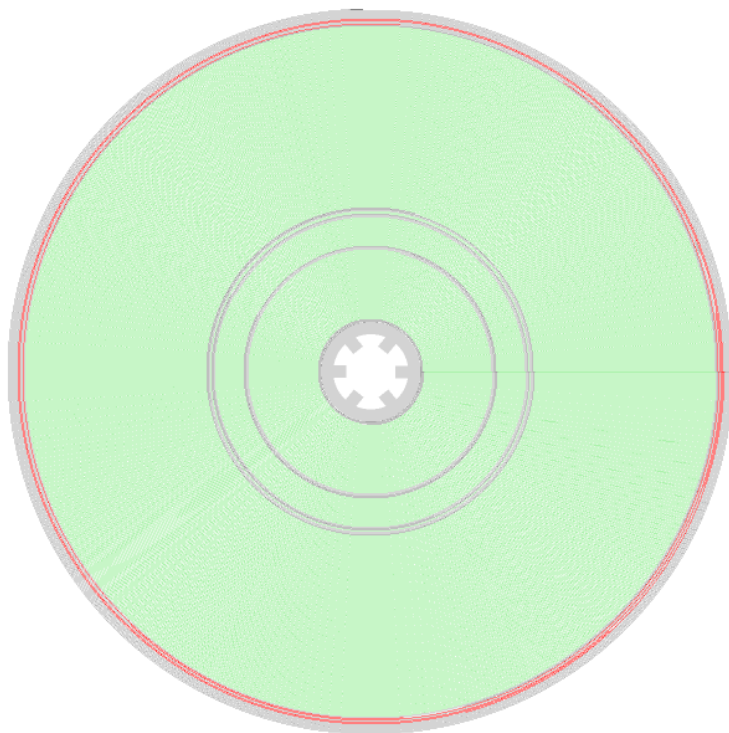


Figure 2.19: Our image of the spool from the start of this chapter but this time with each section colored in as described in the previous image.

When it has finished it has loaded each of the four chunks to the following positions in RAM:

Start Address	End Address	Note
0800	BFFE	The game starts at \$0800 in RAM.
BF00	BFFF	Note this will overwrite the last 255 bytes written above.
C000	CFFE	Third chunk written right up to the start of System ROM at D000.
E000	F7FF	The rest of our data written after System ROM.

Figure 2.20: The four chunks of game data.

To give you an idea of where the different parts of the game end up in the computer's memory let's take a look at where everything goes using the chart below:

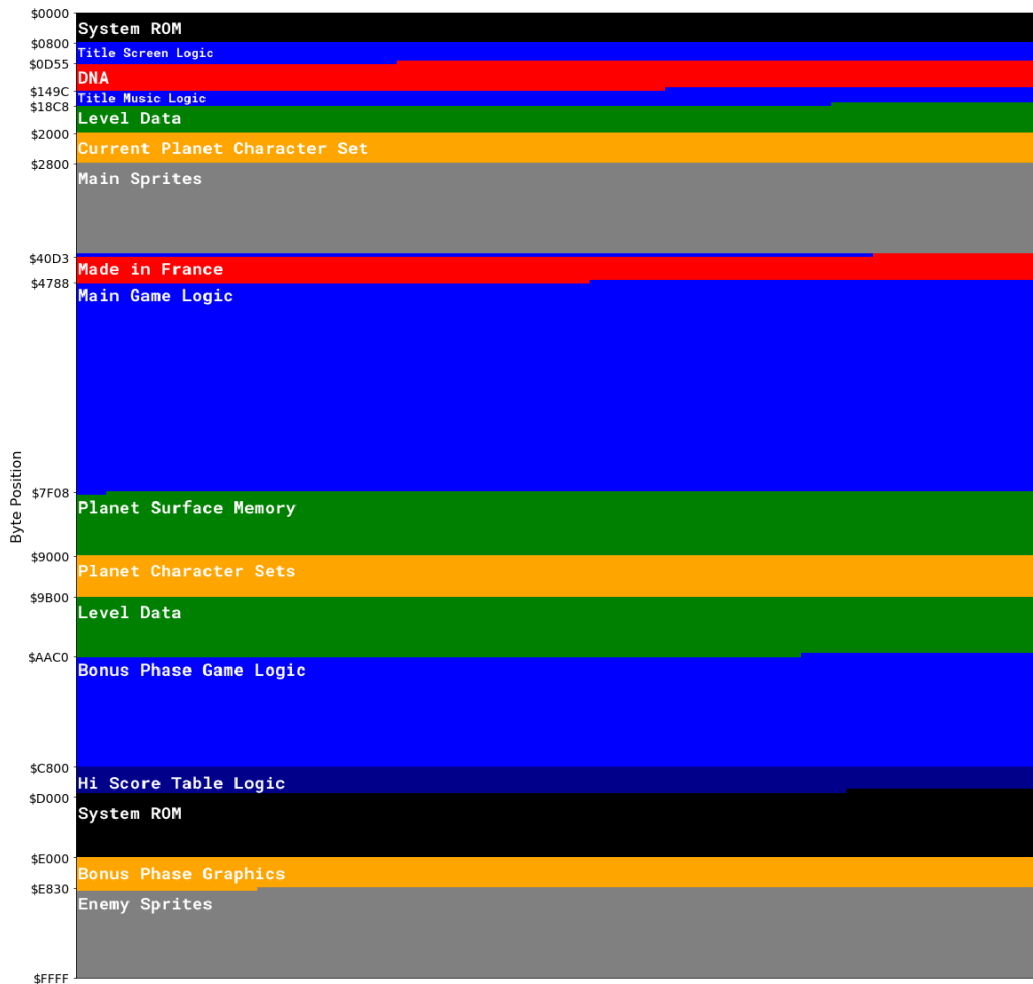


Figure 2.21: Where the different parts of the game end up in memory. Note the sections of 'System ROM' in black. The loader had to avoid copying to these as they are where the C64's operating system lives.

2.4 Putting an End to the Madness

With all the data read and loaded to the C64's RAM you might wonder how the loader knows what to do next (i.e. to run the game) and how it will know where to start running it from. The answer is given in the `Header Data` for the final chunk of data:

Field Description	Field Value	Note
Header Sentinel	01	Indicates the start of the header, expected to be non zero.
Load Address	00 E0	Address to load to: \$E000
End Address	00 F8	End Address to load to: \$F800
Execution Address	10 08	Address to start execution at: \$0810.
Next Action Indicator	00	00 means start executing code, don't read any more data.
Execution Type	02	How to execute the code: 01 means used the address given in 'Execution Address'.

Figure 2.22: The meaning of the Data Header values in the final chunk of data read in by MegaSave.

So according to the header data, the loader should stop reading data now and start executing what has already been loaded, and it should start doing this at address \$0810. When we look at the code that has been loaded to address \$0810, this is what we find:

```

*=$0810
StartExecution
    SEI
    ; Tell the C64 to execute the code at MainControlLoop
    ; the next time an interrupt happens.
    LDA #>MainControlLoop
    STA $0319    ;NMI
    LDA #<MainControlLoop
    STA $0318    ;NMI

    ; Turn off the tape deck.
    LDA #$10
    STA $DD04    ;CIA2: Timer A: Low-Byte
    LDA #$00
    STA $DD05    ;CIA2: Timer A: High-Byte
    LDA #$7F
    STA $DD0D    ;CIA2: CIA Interrupt Control Register
    LDA #$81
    STA $DD0D    ;CIA2: CIA Interrupt Control Register
    LDA #$19
    STA $DD0E    ;CIA2: CIA Control Register A
    CLI
LoopUntilExexcutes
    JMP LoopUntilExexcutes

```

Listing 2.2: The first piece of code that is executed in Iridis Alpha.

This routine does two things: it turns off the tape deck and tells the C64 to execute the code at a different location (MainControlLoop) the next time that it wakes up and wonders what to do. Until then it goes into a loop executing LoopUntilExexcutes over and over again.

The C64 isn't getting out of bed and pondering its future over coffee once a day like

the rest of us. It wakes up hundreds of times every second wondering what it should do next, in a permanent state of fizzing attention. So in a few microseconds time it will inspect the address we stored at \$0318 above, immediately go there, and do whatever it is told. What it will be told is contained in `MainControlLoop`, and with that Iridis Alpha will at last get underway.

I've built you up nicely to expect that we will finally get down to business and see how pixels are thrown on the screen, haven't I? Please console yourself with the picture below before proceeding circumspectly to the next chapter.



Figure 2.23: A prg is born.

Some Disassembly Required

We've reached the point where the game has started to execute. We just saw a snippet of code that turned off the tape recorder and prompted the C64 to run a routine called `MainControlLoop`. Though perhaps a little cryptic (and you shouldn't expect yourself to understand it yet), this code is not exactly what the machine 'saw'. Instead it read and executed something far more puzzling-looking:

```
78A9 408D 1903 A900 8D18 03A9 108D 04DD A900 8D05
DDA9 7F8D 0DDD A981 8D0D DDA9 198D 0EDD 584C 3508
```

Listing 3.1: The first piece of machine code that is executed in Iridis Alpha.

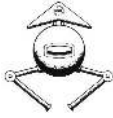
That's right, it executed a stream of bytes. A stream of bytes commonly referred to as *machine code*. Each of these bytes is meaningful to the C64, whether individually, taken in pairs, or even in groups of three. It can comprehend them as instructions to carry out which will shuffle data around in its memory and ultimately result in a game that can be played.

Before we can dig into the internals of how Iridis Alpha works we have to convert all of the machine code we've loaded into memory in the previous chapter into something we have a chance of reading and understanding. This process is called *disassembly* and here we're going to explain how it is done, and along the way gain a little basic understanding of the human-readable language (called '6502 Assembly Language') that we convert the machine code back into.

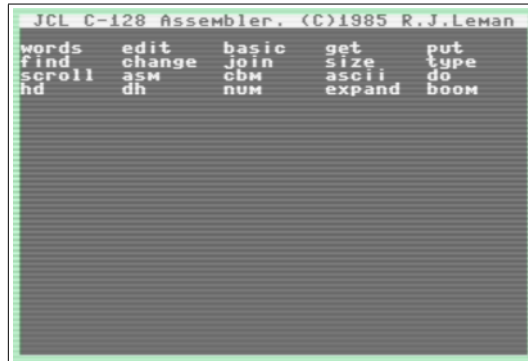
The process is called *disassembly* simply because it is the exact reverse of the process that was originally followed to generate the data on the tape in the first place. That is to say, the data on the tap was generated from the assembly language instructions written by Jeff Minter. Programs that do this conversion from assembly language to machine code are referred to as 'assemblers'. They assemble the instructions written

by the programmer into machine code that the C64 can execute. As self-appointed disassemblers we are going to turn this machine code back into assembly language.

Jeffrey Says



According to Jeff Minter's development diary, Iridis Alpha was programmed and "assembled on a C128 using a partially-finished JCL assembler and the horrible, slow Commodore disk drives." You've read that correctly: Iridis Alpha was not written on a C64 but its more powerful and expensive, and hence much less popular, successor the C128.



The 1984 JCL Assembler on the C128. You can imagine how user-friendly this thing must have been.

This wasn't so uncommon back in the day: computers like the C64 were not powerful enough for getting any actual work done (such as writing and assembling a game), so the programming and assembling was done on more capable, but often prohibitively expensive, microcomputers. As long as the assembler was configured to output a file that could be run on the C64, actual development could be done on any system the developer found tolerable. Or as Minter put it: "Next time I'm gonna use a 6502 X-ASM running in 2.5 Megabytes of RAM on me trusty [Atari] ST..."

Along the way we will have to invent names for things that are meaningful to us. This is because the names Jeff Minter gave to his functions, routines, and variables are long since lost to computer science. They were thrown away by the assembler as unnecessary for execution of the machine code. As we proceed, we will see why, but it hopefully makes sense to say that the 6502 CPU in the C64 doesn't care what things are called. It just cares where in the 65,536 bytes of its RAM things are located. That is to say it only cares about their 'address'.

So let's start by stepping back for a second. Let's put the machine code, and the snippet of code we disassembled it back into, side by side and see what we can learn:

```

78
A9 40
8D 19 03
A9 00
8D 18 03
A9 10
8D 04 DD
A9 00
8D 05 DD
A9 7F
8D 0D DD
A9 81
8D 0D DD
A9 19
8D 0E DD
58
4C 35 08

```

Listing 3.2: Machine Code

```

SEI
LDA #$40
STA $0319
LDA #$00
STA $0318
LDA #$10
STA $DD04
LDA #$00
STA $DD05
LDA #$7F
STA $DD0D
LDA #$81
STA $DD0D
LDA #$19
STA $DD0E
CLI
JMP $0835

```

Listing 3.3: Assembly Language

Straightaway we can infer meanings to some of the bytes:

Byte	Instruction
78	SEI
A9	LDA
8D	STA
58	CLI
4C	JMP

Figure 3.1: Machine code bytes and their assembly language counterparts.

Let's quickly explain what two of these instructions mean as they're extremely common and also fundamental to assembly programming in general.

The instruction `LDA` loads the byte you give it into a small one-byte slot called the 'Accumulator' ('A' for short, so `LDA` is an abbreviation for 'Load to Accumulator'). Think of this slot as a pocket - somewhere for the CPU to store a value for use later on. The technical name for this kind of slot/pocket is a 'register'. The 'Accumulator' is so-called because a lot of the operations performed on bytes stored in this particular pocket have very precise behaviour when addition operations are executed on it. But we never really have to worry about this too often and in the general case it is simply used, as here, as a place to put a value so we can do something else with it.

```
LDA #$40
```

Listing 3.4: Loading the byte \$40 to the Accumulator.

'What we do with it', for the most part in the code above, is go on to store it somewhere else. This is where the STA instruction comes in. This stores the value in the 'Accumulator' pocket at the address in memory that you give to it.

```
8D 19 03
```

```
STA $0319
```

As we can see such addresses are not one byte long, but two bytes. Are they ever more than two bytes long? No, and for a very simple reason. The C64 can only understand addresses that are at most two bytes long and this is what ultimately limits it to 64KB of memory. The largest address it can understand is therefore \$FFFF - the largest value that can be expressed by 2 bytes. Which translates to 65,535. Including zero, this allows us 65,536 bytes, which is 64KB of RAM.

When we look again at the disassembly of the STA instructions we see something quite puzzling:

```
8D 19 03
```

```
STA $0319
```

Shouldn't we have expected 8D 19 03 to translate to STA \$1903 rather than STA \$0319? Why are the numbers back to front like that? The reason is due to something you may have heard described with a word that you've never fully understood and maybe never dared to question. The machine code stores the address 0319 as 1903 because the 6502 CPU in the C64 expects to read addresses with the second half of the number first. When we read numbers we expect it to start with the most significant digits first, i.e. \$0319. But most computer architectures of the 1980s had the opposite expectation - reading the least significant digits first, or the least significant bits first since this is a computer we're talking about, i.e. \$1903. This approach possesses a couple of advantages, the main one being that the computer can start performing an operation on the number (e.g. addition) before it has read all of it. For example, if you're adding 5312 and 2043 you already have enough to get started with once you've read 12 and 43. The carrying etc. can all happen once you've read the rest of the two numbers. This approach is known as 'little-endian' byte ordering. The opposite, which is closer to what we are familiar with when reading numbers ourselves, is called 'big-endian'. If you had never heard of either of those terms before, then you have now.

The next incremental step in understanding our disassembly is to apply some meaning to these two-byte values that we've decoded. As it happens, all the ones given in

this piece of code represent addresses in the C64's memory of \$FFFF bytes, or in the preferred parlance of we humans, 65,536 bytes. Each address here has a particular function, so storing a value in it does something. Let's add the meanings, which are still a little cryptic, to our listing:

```
SEI
LDA #$40
STA $0319
LDA #$00
STA $0318
LDA #$10
STA $DD04
LDA #$00
STA $DD05
LDA #$7F
STA $DD0D
LDA #$81
STA $DD0D
LDA #$19
STA $DD0E
CLI
JMP $0835
```

Assembly

```
SEI
LDA #$40
STA $0319      ;Non-Maskable Interrupt
LDA #$00
STA $0318      ;Non-Maskable Interrupt
LDA #$10
STA $DD04      ;CIA2: Timer A: Low-Byte
LDA #$00
STA $DD05      ;CIA2: Timer A: High-Byte
LDA #$7F
STA $DD0D      ;CIA2: CIA Interrupt Register
LDA #$81
STA $DD0D      ;CIA2: CIA Interrupt Register
LDA #$19
STA $DD0E      ;CIA2: CIA Control Register A
CLI
JMP $0835      ;Jump to address $0835
```

Assembly Language with some comments

As you may remember we said in the previous chapter that this little routine is doing two things: the first is telling the C64 to jump to and execute the routine that starts the actual game the next time it 'wakes up'; the other thing it's doing is turning off the cassette tape reader so that no more data is read from the tape.

The bit that's turning off the tape reader is this:

```
LDA #$10
STA $DD04      ;CIA2: Timer A: Low-Byte
LDA #$00
STA $DD05      ;CIA2: Timer A: High-Byte
LDA #$7F
STA $DD0D      ;CIA2: CIA Interrupt Control Register
LDA #$81
STA $DD0D      ;CIA2: CIA Interrupt Control Register
LDA #$19
STA $DD0E      ;CIA2: CIA Control Register A
```

We are better off treating this series of instructions as a magic formula. The operation of the tape reader is managed by the values stored in a series of bytes between \$DD04 and \$DD0F. The fact that we have to write a variety of values to 5 of them to just stop reading from the tape is a testament to the power of boring overhead - we'll never interact with the tape reader again so studying the entrails here is not going to benefit

us in any way.

The other thing this routine is doing - preparing the C64 to execute the game proper - is more compact and introduces two concepts that are going to be important throughout our tour of the Iridis Alpha code so it is worth spending some time on them here to get them clear.

```
LDA #$40
STA $0319      ;Non-Maskable Interrupt
LDA #$00
STA $0318      ;Non-Maskable Interrupt
```

Listing 3.5: Containing two important concepts.

3.1 Important Concept Number One: High Bytes and Low Bytes

On the face of it these four instructions are doing something very simple. They are storing the value \$40 at address \$0319 and the value \$00 at \$0318. What they are actually doing is storing the address \$4000 in a place where the C64's 6502 CPU will be expected to look in a moment's time and take that as a command to start executing whatever is at address \$4000.

43	36	34	00	40	41	50	45
0315	0316	0317	0318	0319	031A	031B	031C

Figure 3.2: The slice of C64 memory between addresses \$0315 and \$031C and the bytes that live there after we've written \$40 to \$0319 and \$00 to \$0318.

The reversed order we spoke about earlier, the 'little-endian' order, in which the 6502 CPU stores and reads pairs of bytes is observed again here. When the CPU reads the contents of \$0318 and \$0319 it interprets them not as \$0040, which is the order in which they appear to us, but as \$4000.

When discussing this storage arrangement we refer to the contents of \$0318 as the 'Low Byte' and the contents of \$0319 as the 'High Byte'. 'High' is just another way of saying 'first', and 'Low' another way of saying 'second'. When taking about a values like \$4000 stored in \$0318-\$0319 such as in this case \$40 is the 'High Byte' and \$00 is the 'Low Byte'.

Writing values to a pair of adjacent addresses in memory like this so that they can be subsequently interpreted as yet another address to get something from or do something with is a very common pattern in programming 6502 CPUs such as the C64's and we will encounter it a *lot* in this book.

It's a strange sort of indirection when you first attempt to understand it. Instead of storing actual values at an address, we're storing an *address* in the address. If you are familiar with other programming languages you may already recognize this concept and understand how powerful it can be. If you are not it will probably seem strange and maybe even wasteful. Seeing the many uses it is put to in the Iridis Alpha code may persuade you otherwise, but hopefully when we look at the use it is put to here you may begin to get a flavor of its utility. Let's do that by looking at our second important concept.

3.2 Important Concept Number Two: Interrupts

```
LDA #$40
STA $0319      ;Non-Maskable Interrupt
LDA #$00
STA $0318      ;Non-Maskable Interrupt
```

Listing 3.6: It's an interrupt. And it's non-maskable.

We previously waved away what's happening in this code by saying that the address \$4000 will be interpreted as an address to jump to and start executing the next time the 6502 CPU 'wakes up'. That's a lot of hand-waving.

The technical term for this 'waking up' is an 'interrupt'. This waking up happens incredibly frequently, 60 times every second. 60 times a second the C64 will stop what it's doing and execute whatever is given as an address by the bytes at \$0318-\$0319.

The number 60 may ring a bell for you in this context. A common aspiration for graphics-based games, and minimum table-stakes today, is that a game runs at 60 'frames per second'. In other words, that at least 60 times a second the display is updated and whatever is on the screen moves a little bit.

While a C64 programmer would never achieve 60 frames per second in practice, 'interrupts' are the C64's mechanism for at least getting some of the way there. They allow the game developer to at least do something to the display many times per second. Whatever it is, it has to be something short and sweet and at the same time effective enough to actually progress the gameplay. This is why this concept is important to us: most of the important things that happen in Iridis Alpha will be effected during an interrupt. When we look at moving, shooting, and blowing things up, all of them are

going to happen in routines that are called multiple times per second by the 6502 CPU executing the code that has been stored at the address \$0318-\$0319 at that point in time.

And this is our first taste of the power of storing an address in an address. Depending on where we are in the game - be it the title screen, the main game, the bonus phase, or in a pause mode sub-game, the code that we want to run during these interrupts will be different.

We can see this in action if we look at what happens when the code at \$4000 is executed. We've given this address \$4000 a 'label' name of `MainControlLoop` in our disassembled code so `MainControlLoop` always refers to whatever lives at \$4000.

```

MainControlLoop
    LDA #$00
    SEI
p4003    LDA #<MainControlLoopInterruptHandler
    STA $318    ;NMI
    LDA #>MainControlLoopInterruptHandler
    STA $319    ;NMI

```

Listing 3.7: The code at \$4000.

As you can see the first thing it does is change the address of the code that should be executed at every 'interrupt'. It's now an address referred to by the label `MainControlLoopInterruptHandler`. This happens to be address \$6B3E but the beauty of using labels in our code is that we no longer need to worry about the number of the addresses anymore. The label will do the job for us. It does mean we have to know what the syntax `#<MainControlLoopInterruptHandler` means though.

What it means is: if `MainControlLoopInterruptHandler` lives at \$6B3E the `#<` decorators refer to the \$3E part of the address, so we're actually saying `LDA #$3E`, i.e. load the value \$3E into the 'Accumulator'. Similarly the syntax `#>MainControlLoopInterruptHandler` refers to the \$6B part of the address.

43	36	34	3E	6B	41	50	45
0315	0316	0317	0318	0319	031A	031B	031C

Figure 3.3: The values in \$0318 and \$0319 after we've updated them in `MainControlLoop`.

We've almost squeezed as much as we can out of this short and boring snippet of code. There's just one last thing to point out that will be useful to us later. The last instruction in the routine:


```
JMP $0835
```

Listing 3.8: Jump.

This tells the CPU to jump to the address \$0835 and execute whatever is there. What is at \$0835? This:

```
JMP $0835
```

Listing 3.9: Hello again.

That's right - it's jumping back to itself and will execute in an infinite loop repeatedly executing the instruction over and over again. The reason it does this is because it won't be doing it very long. The interrupt will take over and steal execution away to the new interrupt handler we just specified (MainControlLoopInterruptHandler so that the C64 can find better things to do than run in circles.

Before we move on let's take one last look at our disassembly accomplishment.

```
StartExecution
    SEI
    ; Tell the C64 to execute the code at MainControlLoop
    ; the next time an interrupt happens.
    LDA #>MainControlLoop
    STA $0319    ;NMI
    LDA #<MainControlLoop
    STA $0318    ;NMI

    ; Turn off the tape deck.
    LDA #$10
    STA $DD04    ;CIA2: Timer A: Low-Byte
    LDA #$00
    STA $DD05    ;CIA2: Timer A: High-Byte
    LDA #$7F
    STA $DD0D    ;CIA2: CIA Interrupt Control Register
    LDA #$81
    STA $DD0D    ;CIA2: CIA Interrupt Control Register
    LDA #$19
    STA $DD0E    ;CIA2: CIA Control Register A
    CLI
LoopUntilExexcutes
    JMP LoopUntilExexcutes
```

Listing 3.10: Fully disassembled

Hopefully with these basics under our belt we can begin to understand how the interesting parts of Iridis Alpha work.

The First 16 Milliseconds

OK, where were we? At the end of [A Little Archaeology](#) we were just about to get started and begin the race to get something up on the screen. We had set up an interrupt to make us jump to `MainControlLoop` at address \$4000.

Our first act in `MainControlLoop` is to make sure that the next interrupt didn't have us jumping to the start of `MainControlLoop` all over again. So we overwrite the interrupt handler with a new address, `MainControlLoopInterruptHandler`. You will be reassured to learn that, having changed the interrupt handler three times in several milliseconds, we are ready to settle down and stick with `MainControlLoopInterruptHandler` as our interrupt handler of choice for the time being:

```
MainControlLoop
    LDA #$00
    SEI
p4003  LDA #<MainControlLoopInterruptHandler
    STA $318    ;NMI
    LDA #>MainControlLoopInterruptHandler
    STA $319    ;NMI
```

Listing 4.1: The code at \$4000.

Our next act is to call a routine to set up the main title screen:

```
; Display the title screen. We'll stay in here until the
; player presses fire or we time out and go into attract mode.
JSR EnterMainTitleScreen
```

Listing 4.2: In `MainControlLoop`

`EnterMainTitleScreen` does a bit of shuffling of data before it brings us to a routine that does something important. This routine is called `InitializeSpritesAndInterruptsForTitleScreen`

and it does some vital setup for the next 16 milliseconds that determines how we go about getting everything on the screen that we need to get:

```

; Set up the our interrupt handler for the title
; screen. This will do all the animation and title
; music work.
LDA #<TitleScreenInterruptHandler
STA $0314      ;IRQ
LDA #>TitleScreenInterruptHandler
STA $0315      ;IRQ

; Acknowledge the interrupt, so the CPU knows that
; we have handled it.
LDA #$01
STA $D019      ;VIC Interrupt Request Register (IRR)
STA $D01A      ;VIC Interrupt Mask Register (IMR)

; Set up the raster interrupt to happen when the
; raster reaches the position we specify in D012.
LDA $D011      ;VIC Control Register 1
AND #$7F
STA $D011      ;VIC Control Register 1

; Set the position for triggering our interrupt.
LDA #$10
STA $D012      ;Raster Position

```

Listing 4.3: In InitializeSpritesAndInterruptsForTitleScreen

You'll notice we've changed yet another interrupt handler, this time to a routine called `TitleScreenInterruptHandler`. I didn't lie to you earlier though. This is not the same interrupt we settled on above. It is a new and important one that affects how get things on to the screen. The particular type of interrupt it is concerned with is something called a 'Raster Interrupt'. A 'raster' can be thought of as a beam of light that scans across the screen from top to bottom and left to right, painting each pixel on the screen one at a time. It travels so quickly down and across the screen painting pixels that it can do so up to 60 times a second.

As it makes this journey our 'Raster Interrupt' gives us the opportunity to tell it to stop once it reaches a certain position on the screen and allows us to run some code before it resumes again. We can do this as many times as we want along the journey, but each time we interrupt it we have to be quick. If our code takes too long the display will flicker and the content of the screen become inconsistent.

At the very end of the little listing above we set our first interrupt to line 16 (\$10) on the screen:

```

; Set the position for triggering our interrupt.
LDA #$10
STA $D012      ;Raster Position

```

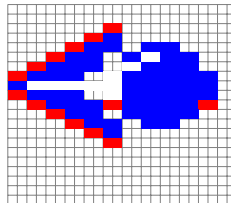
Listing 4.4: In InitializeSpritesAndInterruptsForTitleScreen

This facility is the key that will allow us to do all sorts of magic in the 16 milliseconds it takes to traverse the screen. Every time we get the opportunity to run some code thanks to this interrupt we'll change the location of the screen it should stop at the next time so that we get to stop dozens of times in each single 16 millisecond traversal.

Before we look at how we fit it all in, let's first appreciate just how much we plan to do each time the screen is painted.

4.1 Sprites

The C64 makes 8 sprites available to us. A sprite is a special purpose graphical object that can be up to 24 pixels wide by 20 pixels high. We can place them wherever we want on the screen. They are the core of graphics programming and Iridis Alpha has dozens of them.



GILBY_AIR.LEFT

Figure 4.1: An example of a sprite in Iridis Alpha.

But if the C64 only has 8 sprites, are we limited to displaying just 8 sprites at once on the screen?

The simple answer is that thanks to *Raster Interrupts* we are not: when we run some code after receiving an interrupt we can place new sprites wherever we like in any position that the raster hasn't reached yet. This means our only effective limitation is the number of sprites we can place on a single line, which is eight. If you look carefully at the title screen of Iridis Alpha you'll notice that it is actually split in two.

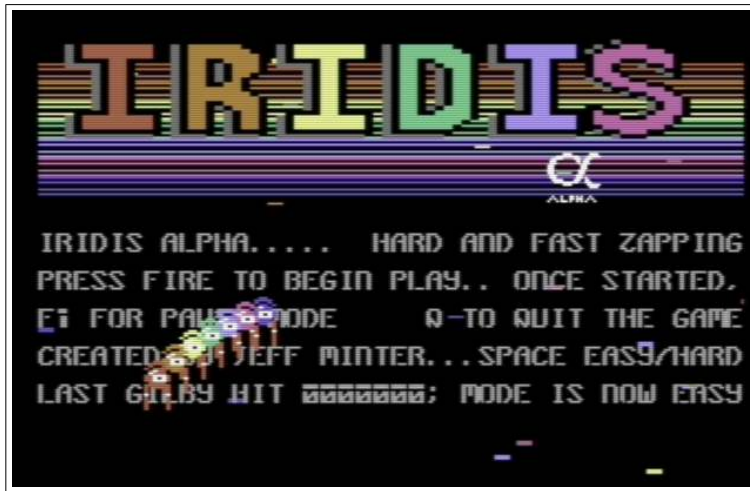


Figure 4.2: The title screen with big letters up top and jumping gilbies down below.

The top half has the title in large letters and the bottom half has a rainbow of jumping gilbies. Each half uses seven sprites (numbered 0 to 6) to display these assets.

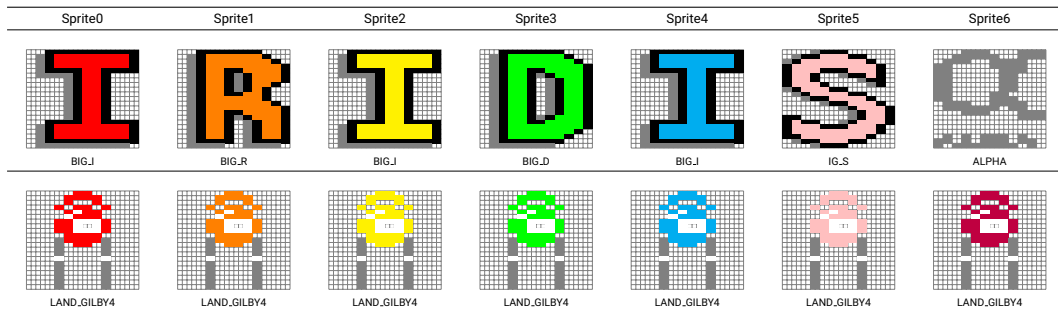


Figure 4.3: The sprites used by the top half of the screen and the bottom half of the screen.

The eighth sprite (Sprite 7) is used on both halves of the screen to display the starfield. This sprite pushes right up against the line limitation. It's painted at intervals throughout the screen but we're careful to avoid it ever being painted twice on the same line. We'll see how this is achieved very soon.

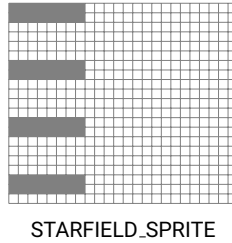


Figure 4.4: The sprite used for painting the starfield. Only a part of the sprite is ever painted!

4.2 Waiting for the Beam

With our 'Raster Interrupt' handler set up as `TitleScreenInterruptHandler` we're ready to react when the raster reaches line 16 on the screen. Since the screen is made up of 512 lines in total this will be along soon.

Before it comes to get us we just have time to prepare the relatively light amount of text we want displayed on the screen in memory. We only need to do this once. Throughout the code we refer to this area we write to as `SCREEN_RAM`. It's an address range between `$0400` and `$07E8`. This is a very simple bitmap representation of the entire screen that is 40 characters wide and 25 characters high, giving a total of 1000 bytes (`$3E8` bytes in hex). If we wanted to think of it as pixels it is 320 pixels wide ($40 * 8$) and 200 pixels high ($25 * 8$). The important thing to remember about this `SCREEN_RAM` is that it is solely for storing what we call character data. You can think of character data as 'text'. This text gets painted first and then sprites get painted on top of it.

In `EnterTitleScreenLoop` we call two routines that will prepare the character data for the raster to paint.

The first, `DrawStripesBehindTitle` writes the rainbow stripes to five lines in the top half of the screen. The second, `DrawTitleScreenText` writes some text to the bottom half of the screen. Before we look at these in detail we need to understand how this thing `SCREEN_RAM` works and how we store characters for display in it.

Our starting point for displaying text on screen is to define what our characters look like. We define the appearance of a character using 8 bytes. This is what the definition of the stripe character looks like:

```

characterSetData
.BYTE $FF,      ; 11111111  *****
.BYTE $00,      ; 00000000
.BYTE $FF,      ; 11111111  *****
.BYTE $00,      ; 00000000
.BYTE $00,      ; 00000000
.BYTE $FF,      ; 11111111  *****
.BYTE $00,      ; 00000000
.BYTE $FF,      ; 11111111  *****

```

Listing 4.5: The 'stripe' character.

As you can see each byte translates to a row of 0s and 1s. Each 1 defines a dot and each 0 a blank space. We end up with a character that is 8 pixels wide and 8 pixels high:

1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1

The stripe character

We create this definition for every character we want to display and store it at the address starting at \$2000 in RAM, which we will give the label `characterSetData`. The order in which we store them determines the reference we use for them later. So for example the stripe character is referred to as \$00, the 'A' character we've defined as \$01 and so on:

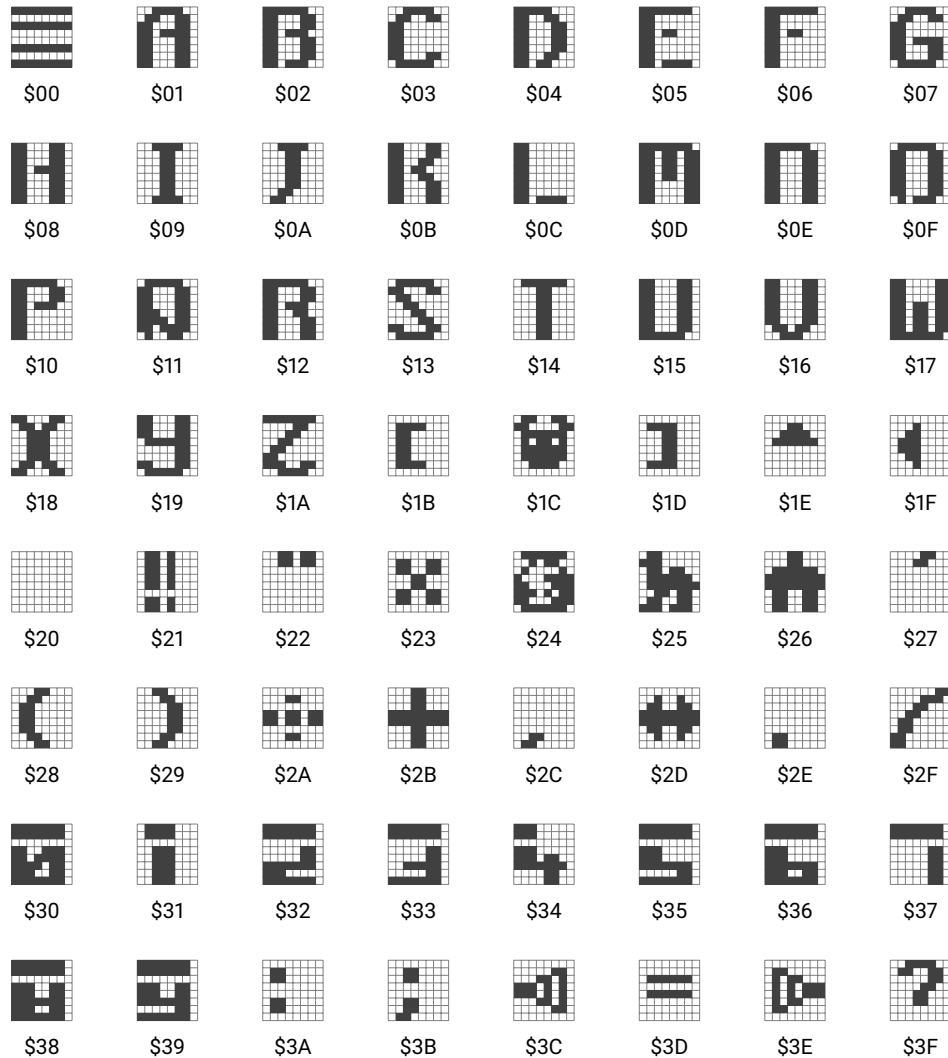


Figure 4.5: Tilesheet: Font Character Set stored at \$2000

With our character set defined in the `characterSetData` at address \$2000 we can now write some text to the `SCREEN_RAM` at address \$0400. For example, when we write a \$01 to a position in `SCREEN_RAM` it will write an 'A' from our character set to that position. Note that when we write a byte to `SCREEN_RAM` we're not yet writing it to the actual screen. This is just a place in memory that the raster (our beam of light) will refer

to later when it is actually writing dots to the screen. So for example, when we write a stripe (\$00) or 'A' (\$01) character index to a particular position in this SCREEN_RAM memory it will know to write it to the corresponding position on the screen.

4.2.1 Drawing the Stripes

So let's use our stripe character at position \$00 in our characterSetData to write some stripes to RAM!

```

DrawStripesBehindTitle
    LDX #40
    LDA #$00
    STA shouldUpdateTitleScreenColors
DrawStripesLoop
    LDA #RED
    STA COLOR_RAM + LINE2_COL39,X
    LDA #ORANGE
    STA COLOR_RAM + LINE3_COL39,X
    LDA #YELLOW
    STA COLOR_RAM + LINE4_COL39,X
    LDA #GREEN
    STA COLOR_RAM + LINE5_COL39,X
    LDA #LTBLUE
    STA COLOR_RAM + LINE6_COL39,X
    LDA #PURPLE
    STA COLOR_RAM + LINE7_COL39,X
    LDA #BLUE
    STA COLOR_RAM + LINE8_COL39,X
    LDA #$00 ; Stripe character
    STA SCREEN_RAM + LINE2_COL39,X
    STA SCREEN_RAM + LINE3_COL39,X
    STA SCREEN_RAM + LINE4_COL39,X
    STA SCREEN_RAM + LINE5_COL39,X
    STA SCREEN_RAM + LINE6_COL39,X
    STA SCREEN_RAM + LINE7_COL39,X
    STA SCREEN_RAM + LINE8_COL39,X
    DEX
    BNE DrawStripesLoop

```

Listing 4.6: Drawing stripes using a loop!

As you can hopefully see, what we're dealing with here is a loop. We load X with the value \$28 (40 in decimal) and perform everything inside DrawStripesLoop until DEX has reduced the value of X to zero.

The magic number 40 gives us a clue that what we are doing in each loop is drawing a character in each column of the screen: remember that our screen is 40 columns wide and 25 rows high. The bit actually writing the stripe character to RAM is:

```

LDA #$00 ; Stripe character
STA SCREEN_RAM + LINE2_COL39,X
STA SCREEN_RAM + LINE3_COL39,X
STA SCREEN_RAM + LINE4_COL39,X
STA SCREEN_RAM + LINE5_COL39,X
STA SCREEN_RAM + LINE6_COL39,X
STA SCREEN_RAM + LINE7_COL39,X
STA SCREEN_RAM + LINE8_COL39,X

```

Listing 4.7: In DrawStripesBehindTitle

For the current column, this writes the stripe character (referenced by \$00 as we mentioned above) to each of lines 2 to 8. The use of the X in the STA statement is an offset. So where X is 14, for example, it will write to the position referred to by SCREEN_RAM + LINE2_COL39 plus 14.

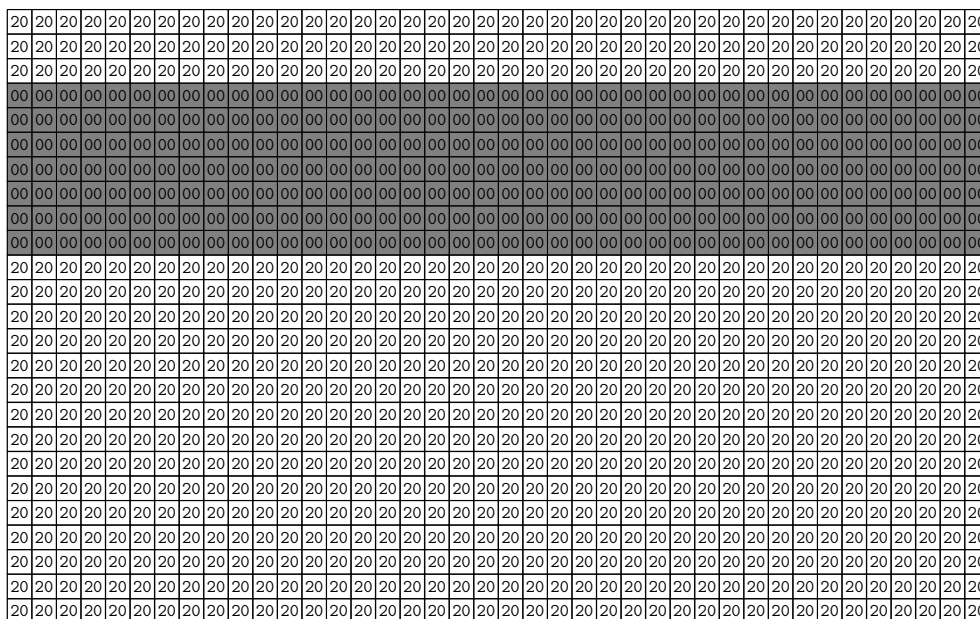


Figure 4.6: The shaded areas of SCREEN_RAM after they have been written to by DrawStripesBehindTitle.

The other thing we do in DrawStripesLoop is set the colors of the stripes. This is achieved using a region of memory similar in concept to SCREEN_RAM, that we call COLOR_RAM. This lives at \$D800 - \$DBFE and is another region of 1000 bytes, each one controlling the color of the character placed at a position in the 40 * 25 character rectangle of our screen.

The part that does this is given below. As you can see it is pretty much the same as the section that writes to `SCREEN_RAM`, but in this instance we load a different colour value to each row.

As we cycle through the loop we add a different colour to each of the 40 columns all 7 rows:

```
LDA #RED
STA COLOR_RAM + LINE2_COL39,X
LDA #ORANGE
STA COLOR_RAM + LINE3_COL39,X
LDA #YELLOW
STA COLOR_RAM + LINE4_COL39,X
LDA #GREEN
STA COLOR_RAM + LINE5_COL39,X
LDA #LTBLUE
STA COLOR_RAM + LINE6_COL39,X
LDA #PURPLE
STA COLOR_RAM + LINE7_COL39,X
LDA #BLUE
STA COLOR_RAM + LINE8_COL39,X
```

Listing 4.8: In `DrawStripesBehindTitle`

We've used a meaningful alias for each of the color values that we write, these are defined as:

BLACK	= \$01
RED	= \$02
PURPLE	= \$04
GREEN	= \$05
BLUE	= \$06
YELLOW	= \$07
ORANGE	= \$08
BROWN	= \$09
LTBLUE	= \$0E

So by writing a value to the corresponding place in `COLOR_RAM`, we're defining the color of the character in that position.

Notice that everywhere else in the the `COLOR_RAM` is set to \$01. This is the value for `BLACK`, reflecting the fact that our background is black by default.


```

DrawTitleTextLoop
    LDA titleScreenTextLine1 - $01,X
    AND #ASCII_BITMASK
    STA SCREEN_RAM + LINE11_COL39,X
    LDA titleScreenTextLine2 - $01,X
    AND #ASCII_BITMASK
    STA SCREEN_RAM + LINE13_COL39,X
    LDA titleScreenTextLine3 - $01,X
    AND #ASCII_BITMASK
    STA SCREEN_RAM + LINE15_COL39,X
    LDA titleScreenTextLine4 - $01,X
    AND #ASCII_BITMASK
    STA SCREEN_RAM + LINE17_COL39,X
    LDA titleScreenTextLine5 - $01,X
    AND #ASCII_BITMASK
    STA SCREEN_RAM + LINE19_COL39,X

    LDA #GRAY2
    STA COLOR_RAM + LINE11_COL39,X
    STA COLOR_RAM + LINE13_COL39,X
    STA COLOR_RAM + LINE15_COL39,X
    STA COLOR_RAM + LINE17_COL39,X
    STA COLOR_RAM + LINE19_COL39,X
    DEX
    BNE DrawTitleTextLoop

```

Listing 4.9: In DrawTitleScreenText

In this case we're not writing a single character over and over, rather we're writing text we've defined elsewhere in variables `titleScreenTextLine[1-5]`:

```

titleScreenTextLine1 .TEXT "IRIDIS ALPHA..... HARD AND FAST ZAPPING"
titleScreenTextLine2 .TEXT "PRESS FIRE TO BEGIN PLAY.. ONCE STARTED,"
titleScreenTextLine3 .TEXT "F1 FOR PAUSE MODE      Q TO QUIT THE GAME"
titleScreenTextLine4 .TEXT "CREATED BY JEFF MINTER...SPACE EASY/HARD"
titleScreenTextLine5 .TEXT "LAST GILBY HIT 0000000; MODE IS NOW EASY"

```

Listing 4.10: In DrawTitleScreenText

In each iteration of the loop we write a character to all five columns, plucking it from the position in `titleScreenTextLine[1-5]` given by `X`.



Figure 4.11: The screen as it would appear after `DrawStripesBehindTitle` and `DrawTitleScreenText` have run. The added grid helps compare with our previous figures for `SCREEN_RAM` and `COLOR_RAM`.

4.3 Racing the Beam

Now we're ready to receive our first beam. You may remember we set this to happen when the raster reached line 16:

```
; Set the position for triggering our interrupt.
LDA #16
STA $D012      ;Raster Position
```

Listing 4.12: In `InitializeSpritesAndInterruptsForTitleScreen`

The job ahead of us is to paint an animated parallax starfield behind the title screen and sprinkle some jumping gilbies along the bottom. To do that we're going to need a data structure that tells us where to paint stuff. For that humble purpose we have `titleScreenStarFieldYPosArray`:

```
titleScreenStarFieldYPosArray .BYTE 72, 78, 84, 90, 96,102,108,114
                                .BYTE 120,126,132,138,144,150,156,162
                                .BYTE 168,174,180,186,192,198,204,210
                                .BYTE 216,222,228,234,240,246
```


This array tells us the lines that we should start a new starfield sprite on. With 30 entries, that's 30 stars on 30 different lines from line 72 on the screen to line 246. If we were to tell the C64 to call our `TitleScreenInterruptHandler` every time it was about to paint one of those lines, we would have a way of drawing one of our starfield sprites on that line. That would also allow us to set the next entry in the list as our next raster position each time we'd finished the current one. We would do this in the same way that we originally set it to 16 above. Working through the list in this way, one interrupt at a time, we could start a new star on all 30 lines in the array.

When we look closely at the routine `TitleScreenStars` we see that this is indeed how the animation is done. To make it our 'interrupt handler' we first set up `TitleScreenInterruptHandler` as follows:

```

; Set up the our interrupt handler for the title
; screen. This will do all the animation and title
; music work.
LDA #<TitleScreenInterruptHandler
STA $0314      ;IRQ
LDA #>TitleScreenInterruptHandler
STA $0315      ;IRQ

```

Listing 4.13: In `InitializeSpritesAndInterruptsForTitleScreen`

We then make `TitleScreenInterruptHandler` call our `TitleScreenStars` routine every time it is ready to handle an interrupt:

```

TitleScreenInterruptHandler
    LDA $D019                ; Has the previous interrupt been
    acknowledged yet?
    AND #$01                 ; If it has..
    BNE TitleScreenStars ; .. then we can handle this one.

```

`TitleScreenStars` works by calling one of three different sub-routines. It picks the one to run depending on which line the raster is on. With `currentStarfieldLine` as in index into our `titleScreenStarFieldYPosArray` it will draw a star on the line in cases where we don't overlap with the gilbies. This means avoiding the lines indexed by the 12th to 16th elements in the array, except when we want to draw the gilbies there with `UpdateJumpingGilbyPositionsAndColors`.

```

TitleScreenStars
    ; For the first 12 lines given by
    ; titleScreenStarFieldYPosArray just paint the
    ; star on that line.
    LDY currentStarfieldLine
    CPY #12                ; Is it 12?

```

```

    BNE MaybeDoStarFieldOrTitleText ; If not, skip.

    ; When the index is 12, we update the position
    ; and color of the jumping gilbies. This is because
    ; the line positions given by elements 12 to 16 in
    ; titleScreenStarFieldYPosArray overlap with them.
    JSR UpdateJumpingGilbyPositionsAndColors

    ; Skip to index 16 for the rest of the stars.
    LDY #16
    STY currentStarfieldLine

MaybeDoStarFieldOrTitleText
    ; If we have a star on the line to paint, do that and return.
    ; DoStarfieldAnimation will update the raster
    ; position for the next interrupt.
    LDA titleScreenStarFieldYPosArray ; Use counter as an index.
    BNE DoStarfieldAnimation
    ; DoStarfieldAnimation will return from the interrupt.

    ; If we got a zero back from titleScreenStarFieldYPosArray
    ; it means we have finished the starfield, so we can get
    ; the title text sprites ready.
PaintTitleTextSprites
    ; Update the X positions of all the stars so that they
    ; move across the screen a bit the next time they're painted.
    JSR TitleScreenMutateStarfieldAnimationData

    ; Reset the line counter.
    LDA #$00
    STA currentStarfieldLine

    ; Reset the next raster interrupt position to 16,
    ; the top of the screen.
    LDA #16
    STA $D012 ;Raster Position

    ; Acknowledge the interrupt, so the CPU knows that
    ; we have handled it.
    LDA #$01
    STA $D019 ;VIC Interrupt Request Register (IRR)
    STA $D01A ;VIC Interrupt Mask Register (IMR)

    ; Update the title text, the sprite colors, the position
    ; of the gilbies, and play a little music.
    JSR UpdateTitleTextSprites
    JSR MaybeUpdateSpriteColors
    JSR RecalculateJumpingGilbyPositions
    JSR PlayTitleScreenMusic
    JMP ReEnterInterrupt
    ; We're done, returns from function.

```

Listing 4.14: TitleScreenStars responsible for choosing what to do at each interrupt.

But whether we're painting a star or a gilby the important thing is, when we're done, to always update the raster interrupt position for the next line that will require an update.

We'll trace this as it actually happens, interrupt by interrupt, and sift through what the `DoStarfieldAnimation` routine does at each step during the raster's first pass at painting the entire screen.

The first time the raster is called, this is what the screen looks like:

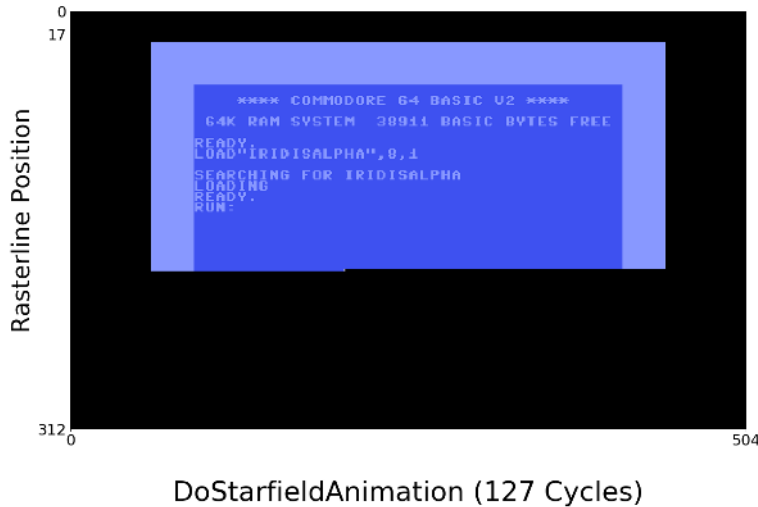


Figure 4.12: The state of the screen the first time the raster interrupt is received at line 16.

Of course we never actually see the screen in this state because it only appears for a microsecond or two, much too fast for us to observe. But as you can see the painting has already started. Everything above line 17 indicated in the figure has been painted black, as per our preparation of `SCREEN_RAM` a little earlier.

What our diagram above also tells us is that in this visit from the beam, `TitleScreenAnimation` chose to execute the sub-routine `DoStarfieldAnimation` and that it took 127 CPU cycles to complete it. Since it takes the raster 63 cycles to do an entire line this means that by the time we've finished this piece of work, the raster will have moved on to the next line - which is why the diagram shows 17 rather than 16. Every time we get an interrupt, the raster doesn't wait for us. We have to work quickly, especially if we're preparing graphics on lines that it's likely to reach soon. This is why each of these subroutines does as little as it can get away with in order to get the job done.

There are three sub-routines the work at each raster interrupt can get divvied out to.

These are `UpdateJumpingGilbyPositionsAndColors`, `DoStarfieldAnimation`, and a cluster of routines starting with `UpdateTitleScreenSpriteColors`. The one that's called the most often is `DoStarfieldAnimation` as it is responsible for sprinkling the screen with animated stars traversing left to right.

The internal accounting responsible for choosing the routine to run is tricky to decipher by just looking at the code. So instead let's follow what actually happens in practice. If we roll ahead to the next interrupt we can already see something happening:

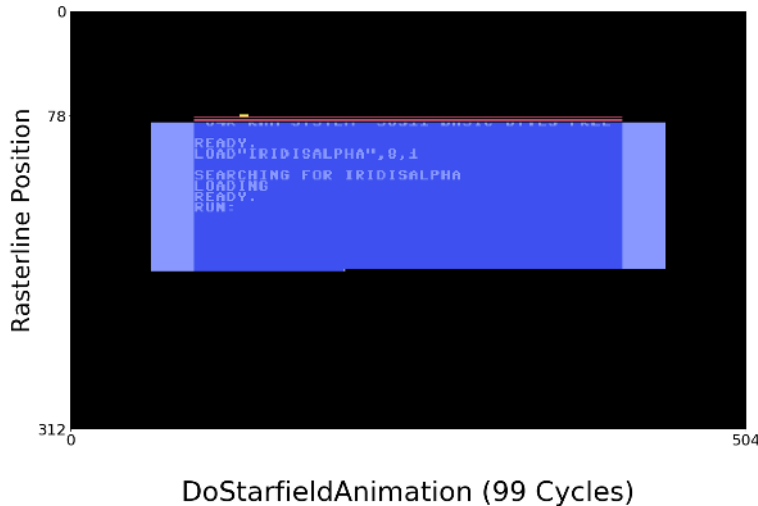


Figure 4.13: The start of the stripes and a star.

If you look closely, you can see a yellow star painted over the first band of red stripes. This is our first sprite. You may be wondering: what about the title sprites? Shouldn't they be there by now? The answer is no: we will paint them when the raster reaches the end of the screen. When it goes to paint the screen a second time (the second 16 milliseconds) they will be ready for painting. We'll see this in action a little later.

So let's see what `DoStarfieldAnimation` did to get this sprite ready for painting.

When `TitleScreenStars` chose `DoStarfieldAnimation` as the sub-routine to run it loaded in a value from `titleScreenStarFieldYPosArray` to the A register:

```
MaybeDoStarFieldOrTitleText
    LDA titleScreenStarFieldYPosArray,Y
    BNE DoStarfieldAnimation
```

Since Y is zero at this point this means it referenced the first value in the array, which

is \$48:

```
titleScreenStarFieldYPosArray .BYTE $48,$4E,$54,$5A,$60,$66,$6C,$72
                                .BYTE $78,$7E,$84,$8A,$90,$96,$9C,$A2
                                .BYTE $A8,$AE,$B4,$BA,$C0,$C6,$CC,$D2
                                .BYTE $D8,$DE,$E4,$EA,$F0,$F6
titleScreenStarFieldXPosArray .BYTE $00,$3A,$1A,$C4,$1B,$94,$7B,$96
                                .BYTE $5D,$4F,$B5,$18,$C7,$E1,$EB,$4A
                                .BYTE $8F,$DA,$83,$6A,$B0,$FC,$68,$04
                                .BYTE $10,$06,$A7,$B8,$19,$BB
```

So when DoStarfieldAnimation is called the first thing it does is set the y-position of the star to paint to \$48 (72 in decimal):

```
DoStarfieldAnimation
    ; A was loaded from titleScreenStarFieldYPosArray
    ; by the caller.
    STA $D00F      ;Sprite 7 Y Pos

    ; Set the X position of the star.
    LDA titleScreenStarFieldXPosArray + $01,Y
    STA $D00E      ;Sprite 7 X Pos
```

Listing 4.15: The start of DoStarfieldAnimation responsible for painting stars.

You can also see it then sets the X position of the star using values plucked from titleScreenStarFieldXPosArray. So we're leaning heavily on these two arrays to decide where to place stars. But so far, so simple. We've placed the star on the screen more or less and when the raster reaches line 72 it will paint it. There's an additional complication to specifying the X coordinate of the star though and we can't really gloss over it here. We'll also encounter this wrinkle elsewhere too so it's worth pausing on for a moment.

The next few lines of the routine do quite a bit of convoluted work to handle something called the spriteMSBXPposOffset of the star. This is our complication.

4.3.1 A Complication

```
    ; Set the rest of the X position of the star
    ; if it's greater than 255.
    LDA titleScreenStarfieldMSBXPposArray + $01,Y
    AND #$01
    STA spriteMSBXPposOffset

    BEQ StarFieldSkipMSB

    LDA #$80
```

```

    STA spriteMSBPosXOffset
StarFieldSkipMSB
    LDA $D010      ;Sprites 0-7 MSB of X coordinate
    AND #$7F
    ORA spriteMSBPosXOffset
    STA $D010      ;Sprites 0-7 MSB of X coordinate

```

Listing 4.16: MSBPosXPos.. some'it.

If you look at the diagram again you may recall we said the screen we're painting is 504 pixels wide. Fortunately the only part we can paint is the section in the center that is 320 pixels wide and 200 pixels high.

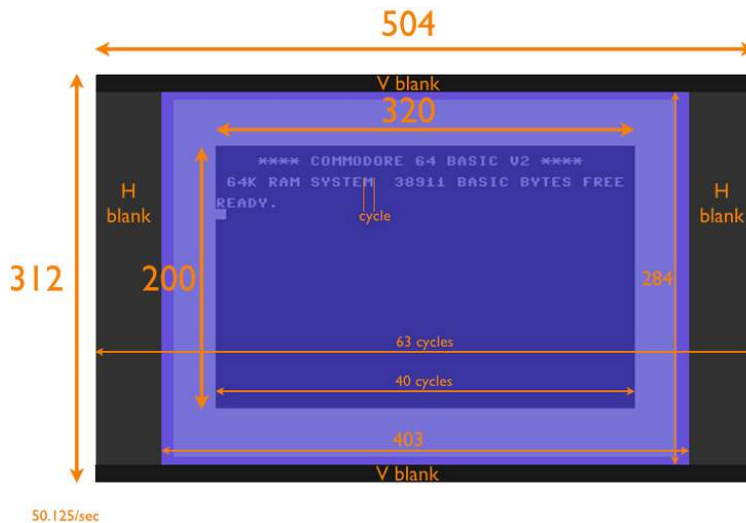


Figure 4.14: The different parts of the screen, we can only paint the bit in the middle. (Source: dustlayer.com)

200 is a value that can be expressed with a single byte. However, 320 is not. A byte can only store a number up to 255 so if we want to specify an X co-ordinate greater than 255 a single byte will not do. The way the C64 works around this is by making a single extra bit for our sprite's X co-ordinate available that brings the available values up from 256 (0 - 255) to 512. Since there are 8 sprites in total we need 8 extra bits to cover this requirement for all of them. For this purpose we use a single byte at address \$D010 that contains the extra bit for all 8 sprites. We refer to this bit as the MSB for the X co-ordinate because it is the 'Most Significant Bit', i.e. the left most bit, in the 9-bit number that we store the X co-ordinate in. That's to say our x co-ordinate is given by combining the value between 0 and 255 we store in our 8-bit byte for 'Sprite 7' in \$D00E and the extra bit we store in \$D010.

Sprite 7	Sprite 6	Sprite 5	Sprite 4	Sprite 3	Sprite 2	Sprite 1	Sprite 0
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	0	0	0	0	0	0	0

The most significant bits in \$D010 for each sprite.

Since we are using 'Sprite 7' for painting the starfield the bit we are interested in is bit 7. The way we're going to manage this value for the starfield is by keeping an array `titleScreenStarfieldMSBXPPosArray` that indicates whether the x co-ordinate for the current index is greater than 255. If the value in there indicates that it is we'll set our bit in \$D010 to 1.

So when we determine from looking in `titleScreenStarfieldMSBXPPosArray` that the x co-ordinate of the star is greater than 256..

```
LDA titleScreenStarfieldMSBXPPosArray + $01,Y
AND #$01
STA spriteMSBXPPosOffset
```

.. we set `spriteMSBXPPosOffset` to indicate that that's the case. That's all this step, with the help of the 'AND #\$01' statement is doing. If 'Bit 1' is set in the value we pluck from `titleScreenStarfieldMSBXPPosArray` it is just an indicator that the x co-ordinate for this star is greater than 256. So if we see a value of \$02 in there our operation 'AND #\$01' will give us a zero result, meaning the intended value of the x co-ordinate is not greater than 256, otherwise it will give us a non-zero result indicating that it is set:

Byte	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
\$02	0	0	0	0	0	0	1	0
\$01	0	0	0	0	0	0	0	1
Result: \$00	0	0	0	0	0	0	0	0

AND'ing \$02 and \$01 gives \$00 (0). For AND to give a 1 both bits must 1 or both must be 0.

This zero result allows `BEQ StarFieldSkipMSB` to evaluate as `True` so we skip ahead to `StarFieldSkipMSB` to set \$D010. It means for us that the value of the x co-ordinate is not going to be greater than 255. If this is not the case, we instead load a value of \$80 to `spriteMSBXPPosOffset` to overwrite the 00 there. This will indicate that the value of the x co-ordinate is greater than 255.

```

    BEQ StarFieldSkipMSB

    LDA #$80
    STA spriteMSBXPathOffset
StarFieldSkipMSB
    LDA $D010    ;Sprites 0-7 MSB of X coordinate
    AND #$7F
    ORA spriteMSBXPathOffset
    STA $D010    ;Sprites 0-7 MSB of X coordinate

```

The remaining step above is to load the value we've arrived at in `spriteMSBXPathOffset` to `$D010`. Since we want to do this without affecting any of the other bits in there that have been set for the other sprites we can't just do a `LDA/STA` as that will overwrite what's already there. The combination of the `AND/OR` operations here accomplishes something quite nifty - it allows us to update just the bit (Bit 7) that interests us in `$D010`.

If we suppose the current value in `$D010` is `$F3`, our `AND #$7F` operation clears 'Bit 7' so that it is always set to zero:

Byte	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
\$F3	1	1	1	1	0	0	1	1
\$7F	0	1	1	1	1	1	1	1
Result: \$73	0	1	1	1	0	0	1	1

AND'ing `$F3` and `$00` gives `$73`. It clears 'Bit 7' for us of whatever value was there originally.

Now when we perform an 'or' operation on the result with '`ORA spriteMSBXPathOffset`' it will have the effect of just setting 'Bit 7' with the value we've stored in `spriteMSBXPathOffset`. In this case it remains at zero because that's what we have in `spriteMSBXPathOffset`, but if we had `$80` in there it would set it to 1:

Byte	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
\$73	0	1	1	1	0	0	1	1
\$00	0	0	0	0	0	0	0	0
Result: \$73	0	1	1	1	0	0	1	1

OR'ing `$73` and `$00` gives `$73`.

4.3.2 Back to the Beam

Now that we've fixed the star's co-ordinates there's just two more things left to do before we're done with handling this raster interrupt. One is to set the color of the star. We do this using a look-up array where we get the color for the star per our current index and set it:

```
LDA titleScreenStarFieldColorsArrayLookUp,Y
TAX
LDA titleScreenColorsArray - $01,X
STA $D02E ;Sprite 7 Color
```

The second, and most important, is that we update the position on the screen that we want the next interrupt to happen. Remember for this visit we were interrupted at line 17. We want to place stars on other lines so we keep a list of the lines we want to write stars on in `titleScreenStarFieldYPosArray`, i.e. an array that stores the y co-ordinates of our stars. What we do is simply update the Raster Interrupt with the next position from this array so that we get called when the raster reaches it:

```
; Update the raster position for the next interrupt
; to the current line - 1. This will allow us to
; draw the sprite multiple times on different lines.
LDA titleScreenStarFieldYPosArray + $01,Y
SEC
SBC #$01
STA $D012 ;Raster Position
```

In this instance we're setting the value to \$48 (72). So when the next interrupt happens it will reveal the star we just prepared. This is the one we took a peek at in Figure 1.10.

The next ten interrupts will continue to revisit `DoStarfieldAnimation`. Let's look at the screen as it unfolds through each of these interrupts:

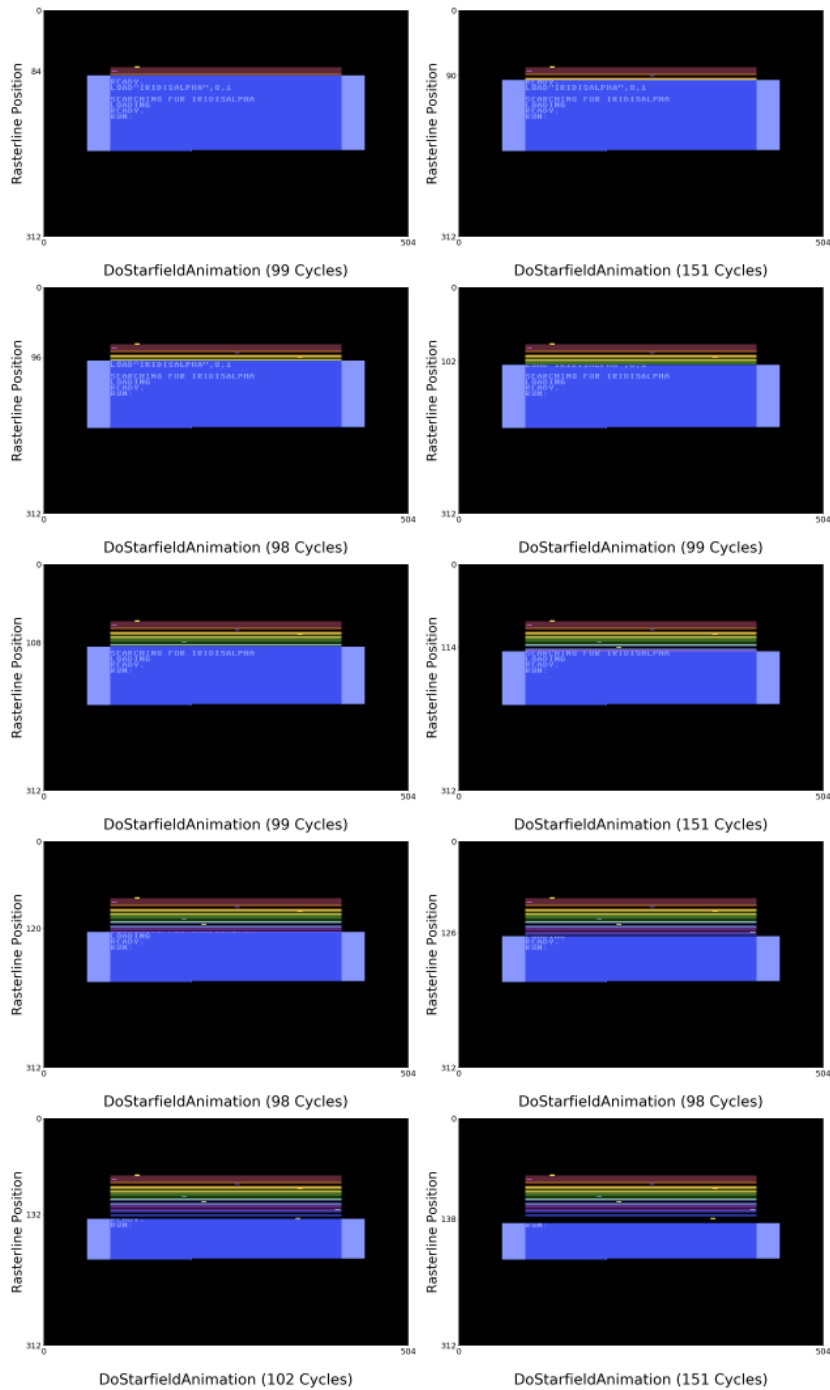
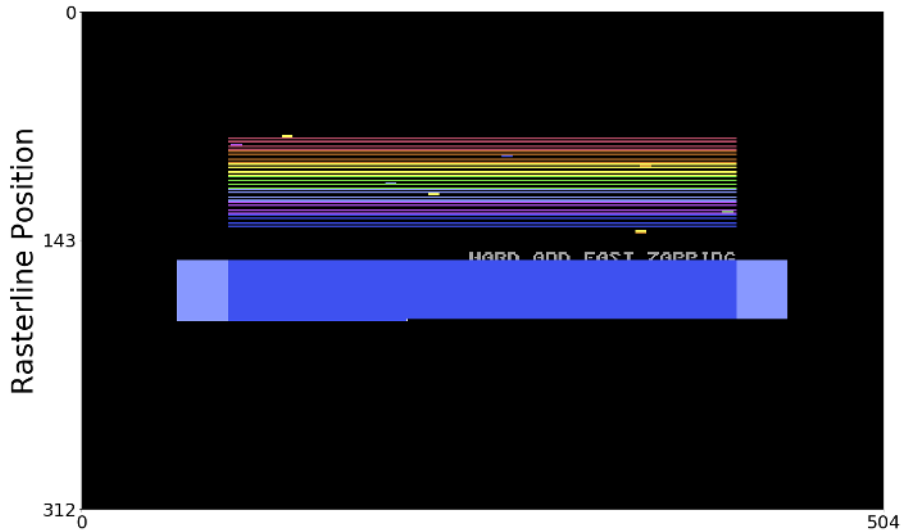


Figure 4.15: The next ten interrupts paint the starfield on the screen until we reach the point at which we want to prepare the gilby sprites.

4.4 Enter The Gilbies



UpdateJumpingGilbyPositionsAndColors (546 Cycles)

Figure 4.16: The point we reach in the screen paint when we decide to prepare the gilby sprites.

Finally we've reached a point in the screen where we're not just going to add another star to the background. We've been keeping a count of the number of interrupts we've handled in `currentStarfieldLine`. When it reaches `$0C` (12) we've handled the raster interrupt twelve times and painted nothing but the text background we prepared earlier and the stars we've added along the way. Now's the time to do something else:

```
TitleScreenStars
    LDY currentStarfieldLine
    CPY #$0C
    BNE MaybeDoStarFieldOrTitleText

    JSR UpdateJumpingGilbyPositionsAndColors
    LDY #$10
    STY currentStarfieldLine
```

The routine we call into here by the name of `UpdateJumpingGilbyPositionsAndColors` will prepare the sequence of jumping rainbow gilbies somewhere in the lower half of the screen before our current raster line. That's why we call it at this point in the raster's journey - because the raster hasn't reached that position in the screen yet (but

will reach it shortly) so now is our opportunity to position the gilbies where we want them.

Since this is an animation sequence we're managing, the approach of `UpdateJumpingGilbyPositionsAndColors` is simply to update their position on the screen. Calculating this new position is something that happens a little later in another routine `RecalculateJumpingGilbyPositions` when we are nearer the bottom of the screen. This means the position values we're picking up here are the initial ones set in the game's code:

<code>titleScreenGilbiesYPosArray</code>	<code>.BYTE \$B2,\$B6,\$BB,\$C1,\$D0,\$C8,\$C1</code>
<code>titleScreenGilbiesXPosArray</code>	<code>.BYTE \$54,\$58,\$5C,\$60,\$64,\$68,\$6C</code>

The next time around we will pick up the positions as re-calculated by `RecalculateJumpingGilbyPositions`. So the positioning of the gilbies and the calculation of the updated positions happen separately. The reason for that approach is simple: there isn't enough time right now to do anything but simply update the positions the gilbies are displayed at. Later on, when the raster has passed line 320 we will have a lot more time available to perform complex calculations because we don't need to worry about the raster painting anything on the screen for a while.

Since there are 7 of them, setting the x and y co-ordinates of the seven gilby sprites is handled by a loop:

```

; Loop through the gilby sprites in the title screen and
; update their position and color
LDX #000
UpdateJumpingGilbiesLoop
    TXA
    ASL
    TAY
    LDA titleScreenGilbiesXPosArray,X
    ASL
    STA $D000,Y ;Sprite 0 X Pos
    BCC SkipGilbyMSBXPath
    LDA $D010 ;Sprites 0-7 MSB of X coordinate
    ORA titleScreenGilbiesMSBXPathArray,X
    STA $D010 ;Sprites 0-7 MSB of X coordinate
    JMP UpdateYPosJumpingGilbies

SkipGilbyMSBXPath
    LDA $D010 ;Sprites 0-7 MSB of X coordinate
    AND titleScreenGilbiesMSBXPathOffset,X
    STA $D010 ;Sprites 0-7 MSB of X coordinate

UpdateYPosJumpingGilbies
    LDA titleScreenGilbiesYPosArray,X
    STA $D001,Y ;Sprite 0 Y Pos

```

```
LDA currentTitleScreenGilbySpriteValue
STA SpriteOPtr,X

; Update Gilby color.
LDA titleScreenColorsArray,X
STA $D027,X ;Sprite 0 Color

INX
CPX #$07
BNE UpdateJumpingGilbiesLoop
RTS
```

Listing 4.17: The loop in `UpdateJumpingGilbyPositionsAndColors` updating the x and y position on screen and color of each of the gilby sprites.

We can see in here the verbosity required to handle the most significant bit of the sprite's x co-ordinate. Just as with the starfield we need a separate array (`titleScreenGilbiesMSBXPoSArray`), in addition to arrays that manage the basic x/y positions themselves.

With the gilbies prepared we fall through and update the starfield again. Once that's done we're finished handling the current raster interrupt. This is followed by another dozen or so interrupts where we again just prepare stars for display and as the raster progresses our gilbies are revealed.

78

4.5 Title Text



UpdateTitleTextSprites and Music (1002 Cycles)

Figure 4.18: We’ve finally reached the bottom of the screen, with gilbies and stars painted, but still no title.

We’ve finally reached the bottom of the screen in our first raster paint, at least the bottom of the portion of the screen that we can paint. When the raster hits line 270 we’re beyond the point that we can place anything on the screen and into the border area. This gives us time to do some more complicated and time consuming stuff.

There’s a relatively full agenda:

```

LDA #$10
STA $D012      ;Raster Position

; Acknowledge the interrupt, so the CPU knows that
; we have handled it.
LDA #$01
STA $D019      ;VIC Interrupt Request Register (IRR)
STA $D01A      ;VIC Interrupt Mask Register (IMR)

; All of this stuff can be done before the raster
; reaches the top of the screen again.
JSR UpdateTitleTextSprites
JSR MaybeUpdateSpriteColors

```

```

JSR RecalculateJumpingGilbyPositions
JSR PlayTitleScreenMusic
JMP ReEnterInterrupt

```

First of all we set the raster interrupt to line 16 at the top of the screen again, then we acknowledge the interrupt. The raster will continue its journey but because we're going to do this while it works its way through the next 42 lines at the bottom of the screen we have more time than at any point previously to get things done.

Adding the title sprites is relatively light work. Just as with the gilbies we use a tight loop to paint each of them on the screen. There's no animation to handle here.

```

titleTextSpriteArray .BYTE $20,BIG_I,BIG_R,BIG_I,BIG_D,BIG_I,BIG_S
...

PaintSpriteLettersLoop
; Assign the sprite.
LDA titleTextSpriteArray,X
STA SpriteOPtr - $01,X

; Shift the value in X left 1 bit and assign to Y.
; So e.g. 6 becomes 12, 5 becomes 10, 4 becomes 8,
; 3 becomes 6 and so on. This allows us to use Y
; as an offset to the appropriate item in $D000-
; $D012 for updating the sprite's position.
TXA
ASL
TAY

; Update the X Position of the sprite
LDA titleTextXPosArray - $01,X
STA $D000 - $02,Y

LDA $D010 ;Sprites 0-7 MSB of X coordinate
ORA titleTextMSBXPosArray,X
STA $D010 ;Sprites 0-7 MSB of X coordinate

; Update the Y position of the sprite
LDA #$40
STA $D000 - $01,Y
DEX
BNE PaintSpriteLettersLoop

```

The other complex thing we do is calculate the next step in the jumping gilby animations. `RecalculateJumpingGilbyPositions` updates the x and y positions of the gilby sprites in `titleScreenGilbiesYPosArray` and `titleScreenGilbiesXPosArray`.

Finally we play a single note from the title music (`JSR PlayTitleScreenMusic`), we cover the music of *Iridis Alpha* in detail in a later chapter.

For now though the raster continues on its journey and progresses through its second paint journey of the screen. The title sprites are finally revealed.



Figure 4.19: The title text is finally revealed

And with that the title sequence is finally up and running after 20 milliseconds or so.

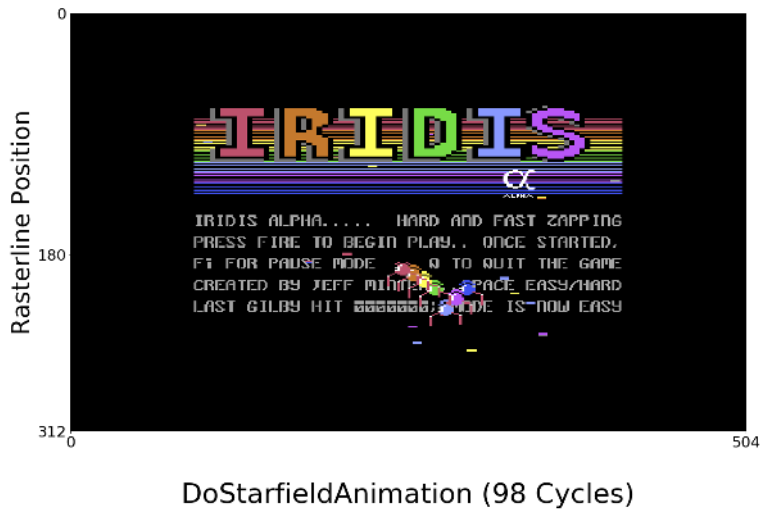


Figure 4.20: We're done here.